



YAŞAR UNIVERSITY

GRADUATE SCHOOL

NON-THESIS MASTER TERM PROJECT

**THE MIXTURE-OF-CONTENTS PROBLEM IN RAG: A TIERED
EVALUATION OF DISCRIMINATIVE RETRIEVAL FOR
CROSS-DOMAIN ATTRIBUTION AND ABSTENTION UNDER
RESOURCE CONSTRAINTS**

CAN ÖZDEN

24500047004

TERM PROJECT ADVISOR: ASSOC.PROF.(PHD) GÖKHAN
DEMIRKIRAN

ARTIFICIAL INTELLIGENCE ENGINEERING

MSC IN ARTIFICIAL INTELLIGENCE ENGINEERING

BORNOVA / İZMİR
MAY 2026

APPROVAL PAGE

We confirm the suitability of the current study as a Non-Thesis Master's Term Project in terms of scope and quality.

Advisor:

Signature:

Assoc. Prof. (PhD) Gökhan DEMİRKIRAN
Yaşar University

.....

Head of the Department:

Assist. Prof. (PhD) Kazım ERDOĞDU
Yaşar University

.....

ABSTRACT

THE MIXTURE-OF-CONTENTS PROBLEM IN RAG: A TIERED EVALUATION OF DISCRIMINATIVE RETRIEVAL FOR CROSS- DOMAIN ATTRIBUTION AND ABSTENTION UNDER RESOURCE CONSTRAINTS

Özden, Can

Non-Thesis Term Project, MSc. in Artificial Intelligence Engineering

Advisor: Assoc.Prof.(PhD) Gökhan DEMİRKIRAN

May 2026

This work addresses cross-domain question answering over corpora that share substantial common vocabulary but differ in underlying semantics, a setting termed the *mixture-of-contents* problem. The chosen domain pair consists of Dota 2 and League of Legends gaming wikis, where shared terms such as *rune*, *jungle*, and *river* carry game-specific meanings that the system must disambiguate. A discriminative retrieval-augmented generation pipeline is proposed, in which each stage scores candidate documents for domain fit and faithfulness rather than leaving disambiguation to the generator. The pipeline consists of hybrid retrieval, a confidence-gated reranker, an 8B-parameter generator, an NLI-based faithfulness check, and a threshold-driven orchestrator that shapes the final answer based on the scores aggregated. The full pipeline runs on a 6 GB-VRAM consumer laptop with no fine-tuning. Evaluation is conducted on a 323-question three-tier benchmark covering clean factual recall, ambiguous cross-domain attribution, and abstention under invalid premises, using RAGAS, manual oversight, and rubric-based grading across twelve ablation variants. On the clean factual tier, the reranker configuration recovers correct answers consistently, producing a clear improvement over no-reranker baselines. On the ambiguous cross-domain attribution tier, ablations indicate that the dominant contribution is the system prompt's explicit attribution enforcement. On the invalid-premise abstention tier, the abstention behavior is driven primarily by threshold-based gating, filtering chunks before they reach the generator. Together, these results suggest

that in resource-constrained multi-domain question answering, attribution accuracy and abstention reliability are shaped more by prompt design and gating thresholds than by retrieval sophistication alone.

Keywords: retrieval-augmented generation, cross-domain question answering, hybrid retrieval, cross-encoder reranking, dynamic context admission, context positioning, abstention, hardware-constrained deployment

ÖZ

RAG'DE İÇERİKLERİN KARIŞIMI PROBLEMİ: KAYNAK KISITLARI ALTINDA ALANLAR ARASI ATIF VE ÇEKİMSERLİK İÇİN AYRIŞTICI BİLGİ GETİRMENİN KATMANLI DEĞERLENDİRİLMESİ

Özden, Can

Yüksek Lisans Dönem Projesi, Yapay Zeka Mühendisliği Yüksek Lisans Programı

Danışman: Doç.Dr. Gökhan DEMİRKIRAN

Mayıs 2026

Bu çalışma, önemli ölçüde ortak kelime dağarcığı paylaşan ancak temel anlamları bakımından farklılaşan derlemeler üzerinde alanlar arası soru cevaplama problemini ele almaktadır; söz konusu durum içerik karışımı (*mixture-of-contents*) problemi olarak adlandırılmıştır. Seçilen alan çifti Dota 2 ve League of Legends oyun wiki'lerinden oluşmakta olup; *rune*, *jungle* ve *river* gibi paylaşılan terimler, sistemin ayrıştırması gereken oyuna özgü anlamlar taşımaktadır. Her aşamasında aday belgelerin, ayrıştırma görevini üreticiye bırakmak yerine alan uyumu ve sadakat açısından puanlandığı ayırt edici bir bilgi getirme destekli üretim (RAG) sistemi önerilmektedir. Sistem; hibrit geri getirme, güven eşikli bir yeniden sıralayıcı (*reranker*), 8B parametrelili bir üretici, NLI tabanlı bir sadakat denetimi ve toplanan puanlara göre nihai cevabı şekillendiren bir orkestratörden meydana gelmektedir. Sistemin tamamı, herhangi bir ince ayar (*fine-tuning*) yapılmaksızın 6 GB VRAM'lı bir tüketici dizüstü bilgisayar üzerinde çalışmaktadır. Değerlendirme; temiz olgusal hatırlama, belirsiz alanlar arası atıf ve geçersiz öncüller altında çekimserlik olmak üzere üç katmanı kapsayan 323 soruluk bir kıyaslama kümesi üzerinde, RAGAS, manuel denetim ve on iki ablasyon varyantı boyunca rubrik tabanlı puanlama kullanılarak gerçekleştirilmiştir. Temiz olgusal katmanda, yeniden sıralayıcı konfigürasyonu doğru cevapları tutarlı biçimde kurtarmakta ve yeniden sıralama içermeyen temel sürümlere kıyasla belirgin bir iyileşme sağlamaktadır. Belirsiz alanlar arası atıf katmanında ablasyonlar, baskın katkının system komutunun (*system prompt*) açık atıf zorunluluğundan geldiğine işaret etmektedir. Geçersiz öncül çekimserlik katmanında ise çekimserlik davranışı, esasen

belge parçalarının üreticiye ulaşmadan önce süzöldüğü eşik tabanlı kapılama tarafından yönlendirilmektedir. Bu sonuçlar bir bütün olarak, kaynak kısıtlı çok alanlı soru cevaplama atıf doğruluğunun ve çekimserlik güvenilirliğinin, yalnızca geri getirme inceliğinden ziyade komut (*prompt*) tasarımı ve kapılama eşikleri tarafından şekillendirildiğini ortaya koymaktadır.

Anahtar kelimeler: bilgi getirme destekli üretim, alanlar arası soru cevaplama, hibrit bilgi erişimi, çapraz-kodlayıcı yeniden sıralayıcı, dinamik bağlam kabulü, bağlam konumlandırma, çekimserlik, donanım kısıtlı dağıtım

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my family for their continuous support, patience, and encouragement throughout this journey. I am also deeply grateful to the broader artificial intelligence research and engineering community. The open sharing of knowledge, tools, and discussions within this community has been instrumental in shaping my understanding and enabling the successful completion of this study. Finally, special thanks are extended to the Dota 2 and League of Legends communities. Beyond their role as gaming ecosystems, they provided the linguistic and conceptual foundation that inspired this work.

Can Özden

İzmir, 2026

TEXT OF OATH

I declare and honestly confirm that my study, titled “THE MIXTURE-OF-CONTENTS PROBLEM IN RAG: A TIERED EVALUATION OF DISCRIMINATIVE RETRIEVAL FOR CROSS-DOMAIN ATTRIBUTION AND ABSTENTION UNDER RESOURCE CONSTRAINTS” and presented as a Non-thesis Master Term Project, has been written without applying to any assistance inconsistent with scientific ethics and traditions. I declare, to the best of my knowledge and belief, that all content and ideas drawn directly or indirectly from external sources are indicated in the text and listed in the list of references.

Can Özden

20.05.2026

TABLE OF CONTENTS

APPROVAL PAGE	iii
ABSTRACT.....	v
ÖZ	ix
ACKNOWLEDGEMENTS	xiii
TEXT OF OATH	xv
TABLE OF CONTENTS.....	xvii
LIST OF FIGURES	xxv
LIST OF TABLES	xxvii
SYMBOLS AND ABBREVIATIONS.....	xxix
1. CHAPTER: INTRODUCTION	1
2. CHAPTER: LITERATURE REVIEW	3
2.1 Large Language Models, Hallucination, and RAG.....	3
2.2 Retrieval Architectures.....	4
2.3 Multi-Domain and Cross-Domain RAG	4
2.4 Cross-Encoder Reranking	5
2.5 Context Positioning.....	5
2.6 NLI-Based Faithfulness	6
2.7 Notable RAG Architectures	7
2.8 RAG Evaluation Methodologies	7
3. CHAPTER: CORPUS CONSTRUCTION	9
3.1 Introduction	9
3.2 Source Acquisition, Preprocessing, and Quality Control	9
3.3 Chunking Strategy and Metadata Schema	10
3.4 Indexing.....	11

3.5	Corpus Statistics.....	12
4.	CHAPTER: METHODOLOGY	15
4.1	Pipeline Overview	15
4.2	Layer 1: Hybrid Retrieval	16
4.3	Layer 2: Cross-Encoder Reranking and Gating Mechanism.....	16
4.4	Layer 3: Generation.....	18
4.5	Layer 3b: NLI-Based Faithfulness	19
4.6	Layer 4: Confidence Orchestration	20
4.7	Context Positioning Strategies	21
4.8	The GPU Shuttling Mechanism	22
5.	CHAPTER: ABLATION SETUP.....	23
6.	CHAPTER: EVALUATION METHODOLOGY	25
6.1	Benchmark Design and Construction	25
6.1.1	Tier 1: Clean Factual.....	25
6.1.2	Tier 2: Cross-Domain Attribution Under Ambiguity	25
6.1.3	Tier 3: Abstention Under Invalid Premise	26
6.2	Evaluation Framework	26
6.3	Tier-1 Evaluation	27
6.3.1	RAGAS Metric Stack.....	27
6.3.2	Manual Oversight.....	28
6.3.3	Retrieval Metrics	28
6.4	Tier-2 Evaluation	28
6.4.1	Rubric.....	28
6.4.2	Scoring	29
6.4.3	Evaluation Example	30
6.5	Tier-3 Evaluation	30

6.5.1	Evaluation Example	31
6.6	Inference on Scores	32
7.	CHAPTER: RESULTS AND DISCUSSION	35
7.1	Headline Results.....	35
7.2	Reranker Contribution.....	38
7.3	System Prompt Contribution.....	38
7.4	Context Positioning.....	39
7.5	Dynamic Gating vs Fixed Top-k.....	40
7.6	Hardware Feasibility and Latency	40
7.7	Discussion	42
8.	CHAPTER: CONCLUSION.....	45
9.	CHAPTER: FUTURE WORK.....	47
9.1	Limitations	47
9.2	Methodological and Evaluation Extensions.....	47
10.	REFERENCES	49
	APPENDICES	51
	APPENDIX 1: MODEL SPECIFICATIONS	51
	APPENDIX 2: HYPERPARAMETERS AND CONFIGURATION	53
	APPENDIX 3: SYSTEM PROMPTS	55
	Vanilla System Prompt	55
	Pipeline System Prompt.....	55
	APPENDIX 4: EVALUATION RUBRIC AND EXAMPLES	57
	Tier-2 Rubric.....	57
	Tier-2 Examples.....	58
	Tier-3 Rubric.....	59
	Tier-3 Examples.....	60

APPENDIX 5: FULL RESULT TABLES	63
--------------------------------------	----

LIST OF FIGURES

Figure 4.1. Pipeline Architecture.....	15
---	----

LIST OF TABLES

Table 3.1. Chunk Amount Distribution.....	12
Table 4.1. Reranker Variants Supported by the Pipeline.....	18
Table 4.2. Aggregated Score Mapping.....	20
Table 5.1. Ablation Variants.....	23
Table 6.1. Query Amount per Tier.....	26
Table 6.2. RAGAS Metrics.....	27
Table 6.3. Tier-2 Rubric.....	29
Table 6.4. Tier-2 Rubric Scoring.....	29
Table 6.5. Partially Correct Example.....	30
Table 6.6. Tier-3 Rubric.....	30
Table 6.7. Fully Correct Example.....	31
Table 7.1. Success Rates per Tier per Variant.....	35
Table 7.2. RAGAS Scores of Tier-1 (in median (mean) format).....	36
Table 7.3. Median Latency per Variant (in Seconds).....	40
Table A1.1. Model Specifications.....	51
Table A2.1. Reranker Configuration.....	53
Table A2.2. Aggregated Score Weights and Answer Tier Bands.....	53

Table A2.3. LLM Decoding Parameters.....	54
Table A2.4. Corpus Chunking and Retrieval Parameters.....	54
Table A4.1. Full Tier-2 Rubric.....	57
Table A4.2. Tier-2 Scoring.....	57
Table A4.3. Fully Correct Example.....	58
Table A4.4. Partially Correct Example.....	58
Table A4.5. Failed Example.....	59
Table A4.6. Full Tier-3 Rubric.....	59
Table A4.7. Fully Correct Example.....	60
Table A4.8. Failed Example.....	60
Table A4.9. Failed Example.....	61
Table A5.1. Success Rates.....	63
Table A5.2. RAGAS Scores for Tier-1. Median (Mean) per-Variant.....	63
Table A5.3. Retrieval Metrics for Tier-1 (means).....	64
Table A5.4. Latency Distribution (in ms).....	65

SYMBOLS AND ABBREVIATIONS

SYMBOLS:

p_1 Success proportion of the 1st variant

σ Standard Error

$w_{R,F}$ Weight of the Reranker (or Faithfulness) Score

ABBREVIATIONS:

agg_score	Aggregated Score
B	Billions
BAAI	Beijing Academy of AI
BGE	BAAI General Embedding
BM25	Best Matching 25
DPR	Dense Passage Retrieval
GB	GigaBytes
GGUF	GPT-Generated Unified Format
GPU	Graphics Processing Unit
HF	HuggingFace
IDF	Inverse Document Frequency
IR	Information Retrieval
JSON	JavaScript Object Notation
LLM	Large Language Model
LM	Language Model
LoL	League of Legends
MOBA	Multiplayer Online Battle Arena
MRR	Mean Reciprocal Rank

ms	milliseconds
MTEB	Massive Text Embedding Benchmark
n_ctx	Context Window Size
NLI	Natural Language Inference
ppt	Percentage points
PR	Primacy-Recency
QA	Question Answering
R	Recency
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
RRF	Reciprocal Rank Fusion
s	Seconds
SE_Diff	Standard Error Difference
T	Temperature
TX	Tier-X
UX	User Experience
VRAM	Video Random Access Memory

1. CHAPTER: INTRODUCTION

Large language models (LLMs) produce fluent text across a wide range of tasks but store their knowledge as a compression of training data, with no link to the sources behind any given claim. Two failure modes follow. A factuality failure is a claim that contradicts the world; a faithfulness failure is a claim that contradicts the evidence supplied to the model at inference time (Ji et al., 2023). Retrieval-augmented generation (RAG) is the response to both: at query time the system retrieves a small set of documents from an external corpus and conditions generation on them, replacing brittle parametric memory with explicit evidence (Lewis et al., 2020).

Most RAG work assumes either a homogeneous corpus or a set of topically disjoint corpora, so that surface vocabulary maps unambiguously onto meaning. That assumption breaks when two corpora share substantial vocabulary but assign different mechanics to the shared terms. The Dota 2 and League of Legends wikis used in this work both contain ward, tower, creep, recall, blink, etc. yet, each pointing to a different in-game entity with different rules. A dense retriever returns a mixture from both domains; a sparse retriever cannot disambiguate the colliding terms; the generator then produces a fluent answer that contains both games’ mechanics. This is the *mixture-of-contents* problem, and it is distinct from standard multi-domain RAG, which presumes topically disjoint corpora with no surface overlap.

This report investigates cross-domain question answering over corpora that share vocabulary but differ in mechanics, under hardware constraints. Dota 2 and League of Legends are used as testing ground; the choice is incidental, and the same pattern recurs in legal corpora across jurisdictions, in medical corpora and across product lines within a single company. Multi-domain RAG (topically disjoint corpora) and cross-lingual RAG are both well-studied; the inverse case, the same surface form pointing to different referents across distinct corpora, has received much less attention.

Four research questions organize the work:

- **Research Question 1:** How much does the pipeline’s individual reliability mechanisms contribute to answer quality and to its ability to signal when an answer is unreliable?
- **Research Question 2:** How does the order of retrieved passages inside the LLM’s prompt affect answer quality?
- **Research Question 3:** Can the system correctly handle queries whose intent is ambiguous between the two domains, and queries with no answer in the corpus, even though no component is dedicated to either task?
- **Research Question 4:** What does a credible local RAG system look like on consumer hardware?

The system is a strictly sequential, non-agentic pipeline. A query runs through hybrid retrieval (BM25 plus a dense bi-encoder, fused with global Reciprocal Rank Fusion), producing a candidate set; a cross-encoder reranker grades each candidate, and a threshold gate discards the low scorers. The surviving passages are placed in the prompt of a local Llama 3.1 8B, which generates an answer. The answer is split into claims, each scored against the admitted passages by a natural-language-inference (NLI) faithfulness checker. A two-signal aggregator combines the retrieval-quality and grounding scores into a single confidence value, mapped to one of four tiers; the lowest tier withholds the answer and asks the user to rephrase.

The contributions are a complete pipeline sized for 6 GB-VRAM local deployment, a 323-question tier-aware benchmark over clean factual, ambiguous cross-domain, and unanswerable queries evaluated with a mix of automated and rubric-based methods, and a twelve-variant ablation isolating the roles of reranking, gating, and context positioning. The scope is deliberately tight: English-only, two domains, factual QA, and a single LLM family, all running on a 6 GB-VRAM laptop. Chapter 2 reviews prior work on retrieval, reranking, context use, faithfulness, and RAG evaluation; subsequent chapters cover the corpus, the pipeline, the ablation study, and the results.

2. CHAPTER: LITERATURE REVIEW

2.1 Large Language Models, Hallucination, and RAG

Large language models (LLMs) store their factual knowledge implicitly in their parameters. A well-documented consequence is hallucination, fluent output that is factually wrong. Ji, Z. et al. (2023) describe two distinct concepts that are often blended in discussions of hallucination: factuality, which assesses whether a claim is correct with respect to the world, and faithfulness, which assesses whether a claim is consistent with the source supplied at inference time. The two criteria are independent: an unsupported claim can be true, and a fully sourced claim can still be wrong if the source itself is wrong. The distinction matters because the two ask different questions: factuality calls for ground-truth comparison, while faithfulness calls for inspection of the evidence.

Two further properties of parametric knowledge of LLMs strengthen the need for an external evidence layer. First, parameters are frozen at the training cutoff. Second, parametric knowledge is skewed toward popular entities: Mallen, A. et al. (2023) report that LLM accuracy on factual questions correlates with entity popularity in pre-training data, with degradation on long-tail entities, and that retrieval augmentation closes much of that gap while scaling alone does not. Their PopQA benchmark is built around this finding. The implication for the present work is direct: specialized wikis (gaming, legal, medical, scientific) sit in exactly the long-tail regime where retrieval is most useful, and the test corpora used here (Dota 2 and League of Legends wikis) are typical of that long-tail regime.

Retrieval-augmented generation (RAG) was introduced by Lewis, P. et al. (2020) as a single-pass framework that conditions generation on a retrieved set of passages. The architecture has since been shaped into a family of pipelines that share a common structure: retrieve, optionally rerank, condition, generate, and sometimes verify. The rest of this chapter walks across that pipeline.

2.2 Retrieval Architectures

Two retrieval paradigms dominate. Sparse retrieval scores documents by weighted lexical overlap and is effective on queries containing rare proper nouns. Dense retrieval embeds queries and documents into a shared vector space and ranks them based on higher dimension positions. Karpukhin, V. et al. (2020) introduced Dense Passage Retrieval (DPR), showing that a learned bi-encoder can outperform a strong lexical baseline on top-k retrieval accuracy across open-domain QA. The dense paradigm has since been further refined later.

Sparse and dense retrievers fail in complementary ways: BM25, a sparse retrieval method, is brittle to paraphrase, while bi-encoders are weaker on exact lexical matches. Hybrid retrieval combines both. The standard fusion mechanism is Reciprocal Rank Fusion (RRF) (Cormack, G. V. et al., 2009), which ranks documents by the sum of their reciprocal ranks across input lists, ignoring raw scores in either retrieval realms. RRF has two practical advantages: it requires no calibration between sparse and dense score scales, and it is empirically robust to noisy or outlier rankings. Its simplicity has made it the default choice in most RAG systems.

2.3 Multi-Domain and Cross-Domain RAG

Three adjacent lines of work are relevant to retrieval from more than one corpus: Multi-domain RAG handles topically distinct corpora where domain-specific terminology and contextual usage differ sufficiently that domain routing is often recoverable from the query. Cross-lingual RAG addresses the inverse problem: a single underlying concept expressed through different forms across languages. Chirkova, N. et al. (2024) evaluate off-the-shelf multilingual retrievers and generators across thirteen languages and document characteristic failure modes for cross-lingual pipelines. The axis of confusion is language, not the shared vocabulary.

A third related lineage is entity disambiguation, originating in Wikipedia entity linking (Hoffart, J. et al., 2011, and the AIDA family). Entity-disambiguation systems resolve a surface mention to one of several candidate referents using contextual features. The task is close in spirit to mixture-of-content that both must decide which of several entities a surface form refers to, but the operation is different. Classical entity-linking systems typically assume a unified knowledge graph or shared identifier space,

whereas cross-corpus terminological collision involves separate knowledge bases with different schemas.

The gap, then, is: two corpora that share substantial vocabulary, assign different mechanics to the shared terms, and respond to the queries through a single retrieval endpoint. To my knowledge, this configuration appears underexplored in current benchmark suites.

2.4 Cross-Encoder Reranking

Bi-encoders embed query and document independently, which makes them fast at index time but blind to any interaction between the two parties. On the contrary, Cross-encoders concatenate the (query, document) pair and run full self-attention before producing a relevancy score, recovering token-level interaction at the cost of being slower.

Cross-encoder outputs are unbounded logits, not probabilities, so they require normalization before they are combined with other signals in a pipeline. Three different transformation choices we have: raw logits (no transformation), bipolar sigmoid (or simply tanh) transformation into $[-1, 1]$, and unipolar sigmoid transformation into $[0, 1]$. The choice depends on how downstream stages expect that value.

A bigger concern is that strong retrievers and rerankers can introduce errors themselves by allowing distracting documents that are topically related to the query but do not contain the answer. Amiraz, C. et al. (2025) formalize a quantitative measure of a passage's distracting effect and report that enabling a reranker can increase the average distracting effect at top positions, because the irrelevant passages a reranker allows tend to be the semantically near-miss cases, and most likely mislead the generator. The implication is that reranking alone is not sufficient when distractors are present, and that a reliability-aware pipeline benefits from an additional gating or filtering signal beyond the reranker's ordering output.

2.5 Context Positioning

Even when relevant passages reach the LLM context, the model does not attend to them uniformly. Liu, N. F. et al. (2024) present that fact as the lost-in-the-middle phenomenon: as the position of the golden passage varies across the input, multi-

document QA accuracy traces a U-shaped curve, with performance highest at the beginning and end of the context and lowest in the middle.

Their scale ablation is directly relevant to the design of the pipeline used. They tested across Llama-2 7B, 13B, and 70B, and observe that the larger models exhibit the U-shape; both primacy and recency anchors are functional, while the 7B model is solely recency-biased; accuracy rises when the gold passage moves toward the end of the context. For the 8B-class generator used here (Llama 3.1 8B), the 7B finding is the nearest evidence point and motivates the recency-biased placement strategy adopted in this work.

Hsieh, C.-Y. et al. (2024) follow up on the U-shape by analyzing attention weights directly and find that the bias is intrinsic: tokens at the beginning and end of the input receive higher attention regardless of relevance. They propose a mechanism, found-in-the-middle, that subtracts the positional component from attention weights. While their mechanism is not adopted here, their analysis inspires a second positioning strategy worth testing: primacy-recency biased placement, in which the highest-scoring chunk sits at position k , the second-highest at position 1, and so on inward. The present work compares the two placements in a controlled ablation study.

2.6 NLI-Based Faithfulness

Natural language inference (NLI) classifies a (premise, hypothesis) pair as entailment, neutral, or contradiction. NLI-based faithfulness checking has been explored through several approaches such as FactCC (Kryściński, W. et al., 2020), the entailment-based scoring of Falke, T. et al. (2019), and the analyses of Maynez, J. et al. (2020), which evaluate whether generated content is supported by its source context. The main limitation in this line of work is that document-level comparisons are often too coarse-grained for the reliable entailment judgement.

The present work adopts a simpler sentence-level decomposition strategy. Instead of evaluating the generated response as a single monolithic unit, the response is split into individual sentences, and each sentence is independently compared against the retrieved context chunks using a zero-shot NLI classifier. This follows the same practical intuition as chunking in retrieval systems: smaller semantic units are easier to evaluate reliably than long heterogeneous passages.

2.7 Notable RAG Architectures

Beyond static pipelines, another work uses self-generated reflection signals to adapt the pipeline’s behavior query by query. Self-RAG (Asai, A. et al., 2024) trains a single language model (LM) to generate special reflection tokens in relation with its output, controlling on-demand retrieval and self-criticism of the generated text. The price is non-trivial: the approach requires additional fine-tuning with custom reflection tokens, requiring a custom training pipeline.

Corrective RAG (CRAG), Yan, S.-Q. et al. (2024), takes a lighter approach. A fine-tuned T5-large evaluator scores the retrieved set and assigns one of three confidence labels: Correct, Ambiguous, or Incorrect, which trigger different downstream actions, including web-search fallback when the corpus is thought to be inadequate. CRAG does not require retraining of the generator model as Self-RAG does.

The common point across this line of works is the use of external scoring signals to gate, redirect, or augment the basic retrieve-then-generate flow. The present work fits the same family but is deliberately lighter than both: no LM training of any kind, no external knowledge fallback, and a confidence output computed as a simple combination of two frozen-model signals.

2.8 RAG Evaluation Methodologies

Evaluating a RAG pipeline involves at least two distinct components: retrieval and generation. Retrieval is typically scored with the following metrics: Hit@k and Mean Reciprocal Rank (MRR). All are computed against gold-passage annotations. Generation side is harder, since two answers can be semantically identical while differing in surface form.

Es, S. et al. (2024) introduce RAGAS, an evaluation framework that scores a RAG output across LLM-judged dimensions such as: Faithfulness, Answer Relevance, Context Recall, and Context Precision.

The present work uses RAGAS metrics for the clean-factual evaluation tier alongside the classical retrieval metrics mentioned above.

3. CHAPTER: CORPUS CONSTRUCTION

3.1 Introduction

The corpora for this work are built from two competitive online game wikis: Dota 2 and League of Legends (LoL). Both titles belong to the multiplayer online battle arena (MOBA) genre and share a common lineage, with the result that their vocabularies overlap heavily such as ward, tower, lane, jungle, carry, and many other terms. However, the underlying mechanics of those terms differ. A ward in Dota 2 has different cost, charge, and counterplay rules from a ward in LoL. The two corpora therefore exhibit the mixture-of-contents problem.

Both games are supported by community-maintained wikis that are freely browsable and structured enough to admit automated extraction, which makes the corpus reproducible without commercial agreements or restricted-access data.

3.2 Source Acquisition, Preprocessing, and Quality Control

Source pages were scraped from Dota 2 Fandom wiki (dota2.fandom.com) and League of Legends Fandom wiki (lol.fandom.com) pages. For each domain the scraping pipeline traverses three layers of information: hero/champion pages, item pages, and general mechanics pages and registers each page into a structured JSON document that mirrors the wiki’s section layout (e.g. profile, basics, abilities, guide, trivia, changelog, etc.).

A pre-processing pass cleans the raw retrieved text from the wikis: **infobox** decoration, and citation markers are stripped; tables are flattened into key-value sequences so that they comply with plain-text embedding; **MediaWiki** and **GroupQuery** template syntax (e.g. double-brace and helper functions) are resolved or removed. The objective is to have a JSON document whose leaves are plain natural-language strings without residual wiki markup, while preserving the section hierarchy that structures each page.

Quality control is performed by a Python checker that scans the JSON outputs for any remaining irregularities such as empty or malformed records, missing required fields, or residual markup that survived the cleaning pass. Flagged lines are manually reviewed and either repaired at source or removed before the JSON outputs are passed downstream.

Both source wikis publish under permissive but non-identical licenses. The Dota 2 Fandom wiki is licensed under CC BY-NC-SA 3.0; the League of Legends Fandom wiki is licensed under CC BY-SA 3.0. The non-commercial clause (NC) on the Dota 2 wiki permits academic use of the present work but avoids commercially intended usage.

3.3 Chunking Strategy and Metadata Schema

Each pre-processed JSON document is converted into retrieval chunks. Per-entity-type chunking walks the JSON tree node by node and applies a node-type-aware splitting rule and falls back to character-level splitting only when a single node exceeds the token budget.

Three entity types are covered per domain: heroes / champions, items, and game mechanics. Within an entity, four node categories are classified. Safe nodes (e.g. basics, changelog) are stable, self-contained, and constructed as a single chunk. List nodes (e.g. abilities) are decomposed item-by-item, with each list element becoming its own chunk. Split nodes (e.g. trivia) are free-form text that must be broken in length. Complex nodes (e.g. guide) have arbitrary nested structure and are traversed recursively until each leaf can be constructed as a chunk.

The chunk-size budget is 512 tokens to accommodate most commonly available embedding models. This constraint brings the decision that a chunk that exceeds 512 tokens cannot safely participate in potential tasks related to embedding without truncation. When a single node's token length exceeds the budget, the chunking function falls back to `RecursiveCharacterTextSplitter` utility of LangChain with a 50-token overlap so that sibling splits share enough context to remain individually meaningful.

Every chunk carries a fixed metadata schema: game, hero / champion / item / mechanics, etc. The fields are also injected into the chunk text as prefix so that the

embedding model and the lexical retriever both see explicit structural context without having to recover it from the chunk's text body. This way, intra-chunk relationships are aimed to be strengthened. Furthermore, `chunk_index` field links sibling splits of an oversized node mentioned at the end of the previous paragraph, preserving intra-chunk coherence across the boundary together with the 50-token overlap. At retrieval time only `game` key is consumed as a filter; the remaining fields are reserved for future use of metadata-aware retrieval which is flagged for future work in Chapter 9.

As a concrete example, the `basics` node of the Dota 2 hero Abaddon produces a single chunk, with the corresponding metadata record shown alongside:

```
page_content:
  Game: Dota 2 | Hero: Abaddon | Path: basics | Data:
  {"attributes": {"primary_attribute": "Universal",
    "strength": 22, "agility": 23, "intelligence": 19,
    "health": 604, "mana": 303, ...}, "combat_stats":
    {"attack_damage_min": 2, "attack_damage_max": 12,
    "armor": 3.8, "movement_speed": 325, ...}}
```

```
metadata:
  game      : "Dota 2"
  hero      : "Abaddon"
  primary_node: "basics"
  sub_node  : "root"
  full_path  : "basics"
  chunk_index : 0
```

Both the scraping pipeline and the chunker source code are publicly available in the project repository (<https://github.com/c-ozd/term-project>)

3.4 Indexing

Two retrieval indices are built over the chunk corpus. The dense index is a single unified ChromaDB collection that stores embeddings from both domains, with the game field carried as chunk metadata. Embeddings are produced by the Qwen3-Embedding-4B model quantized to Q8. The model was selected because it ranks strongly on the public MTEB leaderboard while still fitting the 6 GB-VRAM budget after Q8 quantization; alternative models that exceed the VRAM budget and requiring other than zero-shot capabilities were excluded.

The sparse index is built differently. Rather than a single BM25 index over the combined corpus, two separate indices (`bm25_dota2` and `bm25_lol`) are constructed, one per domain, with inverse document frequency (IDF) computed independently

within each. Computing IDF within each domain preserves the term’s lexical weight relative to the rest of its own associated corpus.

The two index types are therefore asymmetric by design: a unified dense store and two partitioned sparse stores. At query time, results from the BM25 indices and the dense index are combined using global Reciprocal Rank Fusion across all four returned lists (`dense-Dota 2`, `dense-League of Legends`, `bm25_dota2`, and `bm25_lo1`).

3.5 Corpus Statistics

The resulting corpora contain 11,567 chunks drawn from 802 distinct entities (376 in Dota 2; 426 in League of Legends). Table 3.1 reports the per-domain and per-entity-type breakdown.

Table 3.1: Chunk Amount Distribution

Domain	Entity type	Chunks
Dota 2	Heroes	4,053
Dota 2	Items	1,227
Dota 2	Mechanics pages	194
LoL	Champions	3,912
LoL	Items	1,412
LoL	Mechanics pages	769

Chunk lengths sit comfortably below the 512-token budget. Across the full corpus, the mean chunk length is 239 tokens, the median is 212 tokens, and the 95th percentile is 463 tokens; the longest single chunk is 506 tokens. The minimum chunk is 25 tokens, which corresponds to a short safe node (typically a small profile or changelog record). Per-domain length distributions are similar (Dota 2 mean 228, median 208; LoL mean 249, median 216).

A few coverage limitations are worth recording. First, the corpus is a snapshot: patches, balance changes, and new item or hero/champion releases that post-date the scraping

pass are not reflected (Scraping Date: 09.02.2026). Second, the mechanics-page coverage is intentionally selective rather than exhaustive; broad game-state pages (lane mechanics, vision, structures) are included, but auxiliary content such as cosmetic items, esports event pages, and developer interviews are excluded. Third, both wikis carry community-supplied guide and trivia sections whose accuracy depends on editor diligence; the present work treats wiki content as authoritative for retrieval purposes without independent verification of every claim.

4. CHAPTER: METHODOLOGY

4.1 Pipeline Overview

The system is a strictly sequential, non-agentic discriminative pipeline. A query enters Layer 1 (hybrid retrieval), passes through Layer 2 (cross-encoder reranking and gating), its corresponding answer is generated in Layer 3 (LLM), it is grounded-checked by Layer 3b (NLI faithfulness), and finally orchestrated into a tiered response by Layer 4 (confidence orchestration). Each layer either admits, re-scores, transforms, or measures the output of the previous one, and no layer queries the other layer(s). Full data flow is shown in Figure 4.1. (See Appendix 1 for full list of models used.)

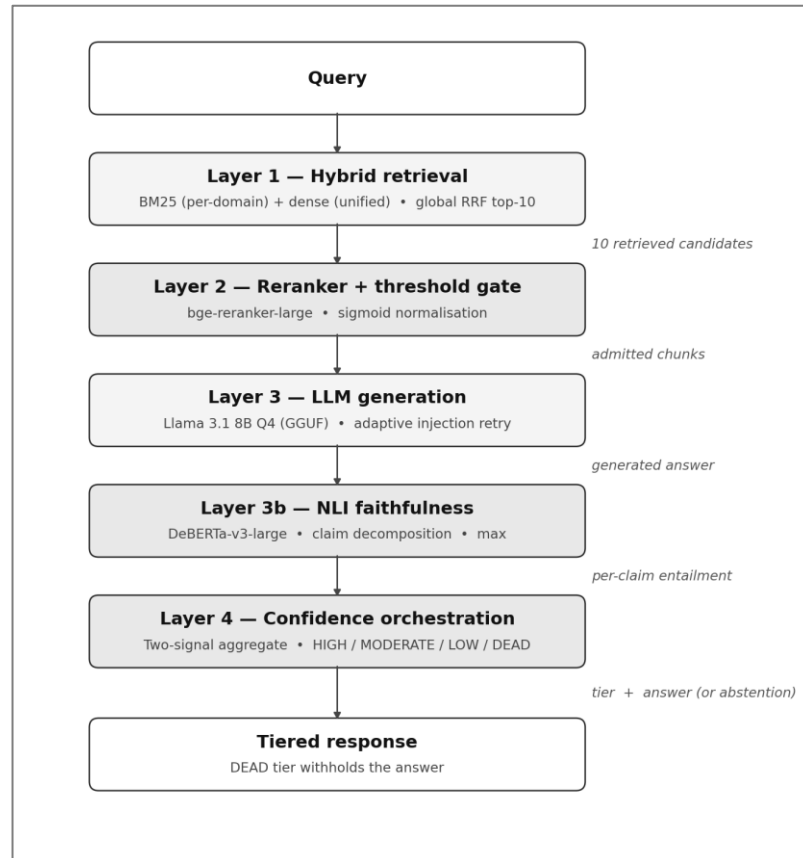


Figure 4.1: Pipeline Architecture

The design philosophy separates two roles. Layer 2 is the bouncer: it admits or rejects chunks at the door of the LLM’s context window. Layers 3b and 4 are graders: they

measure how well the generated answer is grounded in the admitted evidence and presents that measurement as a confidence tier. For the three top tiers (HIGH, MODERATE, LOW), the answer is shown to the user along with the tier label; only the lowest tier (DEAD) withholds the answer and asks the user to rephrase.

4.2 Layer 1: Hybrid Retrieval

Layer 1 processes four ranked-list queries in parallel: dense and sparse over each of the two domains, fuses the results, and forwards the top candidates to Layer 2.

The sparse path uses BM25 over two domain-partitioned indices (`bm25_dota2` and `bm25_1o1`), with IDF computed independently within each. Documents are tokenized by a simple lowercased-alphanumeric splitter; no stemming or stop-word filtering is applied in terms of tokenization.

The dense path embeds the query with the embedding model and searches through the single unified ChromaDB collection. The unified collection is filtered at query time by the `game` key so that each of the two dense calls returns only candidates from their own domain.

The four retrieval calls are dispatched in parallel via `ThreadPoolExecutor` with four workers. Then, the four retrieval lists are combined by Reciprocal Rank Fusion (RRF) (Cormack, G. V. et al., 2009). RRF ranks each candidate chunk over all input lists in which it appears, ignoring raw scores of BM25 and semantic similarity. The four lists are accumulated into a single global RRF call, and then the top ten candidates are taken globally. A document that appears in multiple lists gains a consensus rank boost.

The output of Layer 1 is a list of ten `RetrievedChunk` objects carrying chunk content, source domain, and rank-fusion score.

4.3 Layer 2: Cross-Encoder Reranking and Gating Mechanism

Layer 2 re-scores the ten candidates with a cross-encoder and decides which of them reach the LLM’s prompt. The cross-encoder used is `bge-reranker-large` (HF Identifier: BAAI/bge-reranker-large), a reranker model that consumes the (query, passage) pair through full self-attention and produces a single relevance logit. An earlier alternative, `jina-reranker-v3` (HF Identifier: jinaai/jina-reranker-v3), was evaluated as well, but rejected because the empirical range of its sigmoid-normalized

scores was too compressed to support reliable tier banding downstream. It is important to have an actionable interval because a one-turn pipeline with no follow-up correction needs a score signal that spreads meaningfully across the useful range, and BGE’s logit distribution does that without further calibration.

The reranker’s tokenizer has a 512-token combined-input budget for (query, passage) pairs. Chunks that would exceed the budget were split with overlap so that a single chunk and query pair can be scored. It should be emphasized that when the splitting is required, the query is paired with every chunk-fragment (window) separately, and the maximum score of them is accepted as the chunk’s reranker score.

Raw logits from the cross-encoder are unbounded. They are normalized to $[0, 1]$ by a temperature-scaled unipolar sigmoid: Unipolar (rather than bipolar, or tanh) is chosen so the reranker signal lives in the same $[0, 1]$ interval as the faithfulness score, which makes the two signals combinable as a sum in Layer 4 without further rescaling. The temperature parameter controls the steepness of squashing the values between 0 and 1; the chosen value ($T = 4$) spreads the typical logit range of **bge-reranker-large** across the useful interval of the gate threshold and the tier bands.

The normalized score serves two purposes: as an ordering signal, it determines which chunks are ranked highest, and as an admission gate, it filters chunks by a minimum-score threshold (**gate_threshold** = 0.45 by default, yet it can be customized per user’s preference). The gate converts the reranker from a sorting step into a filtering step and controls the part of the retrieval-quality decision. If zero chunks clear the threshold, an Option-C fallback allows the single highest-scoring chunk anyway; its low score propagates through Layer 4 and naturally routes the response to the LOW or DEAD tier, conservatively. The naming convention for Option-C comes after the two latter options A and B. Option-A is simply the happy-path, and Option-B halts the process when there is no chunk surviving the gate. For the sake of operational continuity, Option-C is accepted as the flow component.

The pipeline supports four reranker behaviors, listed in Table 4.1. The baseline behavior (Dynamic Top-k) uses the threshold gate, and all the other three are ablation variants studied in Chapter 5.

Table 4.1: Reranker Variants Supported by the Pipeline

Behavior	Scoring	Admission rule
Dynamic Top-k (baseline)	reranker logits, sigmoid-normalized	Admit chunks with score $\geq$ gate threshold (Option-C fallback if none clear it)
Fixed Top-k	reranker logits, sigmoid-normalized	Admit the fixed top-k chunks, ignoring the threshold
No-Reranker	Layer 1 RRF scores	Admit all RRF candidates
No-Reranker Top-k	Layer 1 RRF scores	Admit the first k chunks in RRF order

After admission, the surviving chunks pass through token budget check which trims the assembled context to at most 3,500 tokens so that, together with the system prompt and the query, the assembled input fits comfortably inside the LLM’s context window. Trimming is rank-aware: lower-scoring admitted chunks are dropped first if the budget is exceeded. Another separate guard rejects queries that exceed 150 tokens; such queries would leave insufficient room for the passage side of the reranker’s 512-token input budget, and reranking is skipped entirely in that case.

4.4 Layer 3: Generation

Generation is handled by Llama 3.1 8B Q4 GGUF running via `llama-cpp-python`. The model is loaded with `n_ctx` = 5000 (large enough for the 3,500-token context budget, the system prompt, the query, and a buffer for generation), with all transformer layers offloaded to GPU when available. The model uses temperature 0.5, a fixed random seed of 42 for reproducibility, and a 512-token generation cap.

The augmentation has three parts. A system prompt identifies the dual-domain setting, instructs the model to answer only from the provided context, and carries a cross-domain interaction rule that prohibits constructing relationships, comparisons, or counterfactuals between Dota 2 and League of Legends entities under any

circumstance. The rule is re-stated in several ways (no synergies, no counters, no inverted contrasts, no hedged conditionals) because the colliding-vocabulary is precisely the case in which an LLM would otherwise invent plausible cross-domain claims. The system prompt also includes a context-format guide explaining the chunk-prefix schema. What follows the system prompt is the user query. In fact, the user query is placed both before and after the context block so as to exploit attention zones in favor of the user query. (See Appendix 3 for prompts)

Two run-time refinements are applied. First, an entity-mismatch detector checks whether the admitted context references an entity the user did not ask about in the query; when this is the case, the LLM frames the entity as an example to be presented in the answer rather than as the focus of the question. Second, on rare abstentions caused by the reranker scoring a potentially valuable chunk just below the gate threshold and filtering it out, an adaptive injection-retry mechanism re-injects that highest-scoring gated-out chunk into the context and triggers a single regeneration; only the final answer (attempt 1 or 2) is forwarded to faithfulness scoring. When the retry succeeds, that is the abstention is mitigated with the injection, Layer 4 reads the median of the admitted chunks rather than the mean, otherwise injecting a below-threshold chunk would pull the mean toward zero and distort the aggregate score that Layer 4 uses for tier assignment. No further retries are conducted as it would otherwise distort the logic of gating mechanism.

4.5 Layer 3b: NLI-Based Faithfulness

Faithfulness verification answers a different question from generation: not what should the answer be? But is the answer the LLM produced supported by the evidence it was given? Layer 3b answers this with a frozen natural-language-inference (NLI) classifier.

The model is `deberta-v3-large-zeroshot-v2.0` (Laurer, M. et al., 2023). The generated answer is first decomposed into claims by a sentence-boundary splitter (regex over sentence-final punctuation) and filtered for substantive content (≥ 3 tokens per claim). Each claim is scored against every admitted chunk as a (premise, hypothesis) pair, with the chunk as premise and the claim as hypothesis. Truncation happens when the pair exceeds the context window of the model. The process is identical with truncation in Layer 2 when (query, passage) pairs exceed the context window of the embedding model.

For each claim, the per-pair entailment probability is taken against every admitted chunk, and the claim’s faithfulness score is the maximum across chunks. The answer-level faithfulness signal that flows to Layer 4 is the maximum across claims. Three aggregation choices were considered. **Minimum** was rejected because LLM outputs typically contain filler sentences (greetings, generic closers, hedging phrases) that are not informative, hence, a single weak filler claim would pull the minimum to near zero on otherwise well-grounded answers. **Mean** was rejected because it dilutes the contribution of the one or two informative claims that carry the answer’s actual content. **Maximum** is robust to this filler content imbalance characteristic and is chosen as the aggregation variant.

4.6 Layer 4: Confidence Orchestration

Layer 4 combines two signals from the upstream stages into a single confidence value and maps that value to a response tier. The signals are the median reranker score of the admitted chunks from Layer 2 (a retrieval-quality proxy) and the maximum claim-level faithfulness from Layer 3b (a grounding proxy). Both signals live in $[0, 1]$. The aggregated score is the direct combination:

$$agg_score = w_R \cdot Score_{reranker} + w_F \cdot Score_{faithfulness} \quad (1)$$

$$where\ w_R = 0.70\ w_F = 0.30\ (default\ setting)$$

The default weights are themselves a deployment choice. Setting w_R higher (as in the 0.70/0.30 default) prioritizes retrieval and setting w_F higher would prioritize grounding; the system would be most confident when each substantive claim is strongly entailed, even at the cost of weighting evidence quality less. The weights chosen here to serve as a default and are not claimed to be optimal.

The aggregate score maps to one of four response tiers as shown in Table 4.2:

Table 4.2: Aggregated Score Mapping

Tier	Aggregate score	Response shown to user
HIGH	> 0.70	Answer with source citations

Table 4.2 (continued): Aggregated Score Mapping

MODERATE	> 0.50	Answer with source citations and an uncertainty note
LOW	≥ 0.20	Answer with source citations and a low-confidence warning
DEAD	< 0.20	Answer withheld; user prompted to rephrase

The tier banding is a user experience (UX) feature designed to serve as reliability signal to the end user, not an evaluation metric. The thresholds are conservative, most queries with adequate retrieval and reasonable grounding land in HIGH or MODERATE; the LOW band captures the long tail of usable but uncertain answers; DEAD is reserved for cases in which both signals are weak, in which the responsible action is to abstain rather than to stream a fluent but unsupported answer. Each tier corresponds to a different failure mode that the user is best served by knowing about.

4.7 Context Positioning Strategies

Context positioning is treated as a separate methodology axis that changes the order in which admitted chunks are placed in the LLM’s prompt, holding everything else constant. The axis is motivated by two findings from the literature. Liu, N. F. et al. (2024) report the lost-in-the-middle effect: for long-context LLMs, accuracy on multi-document QA shows a U-shaped curve as the position of the gold passage changes, with the strongest attention at the beginning and end of the context. Their scale ablation is also notable: Llama-2 7B exhibited a recency bias rather than the U-shape, with the primacy effect suppressed or absent at 7B but recovering at 13B and 70B. For the 8B-class model used in this work, the 7B finding is the nearest published evidence point. Hsieh, C.-Y. et al. (2024) follow up by analyzing attention weights directly and confirm that the positional bias is intrinsic to the model’s attention rather than a property of any particular task. Therefore, two placement strategies are studied.

Recency-biased placement orders admitted chunks linearly in ascending score: the lowest-scoring admitted chunk positions first in the prompt and the highest-scoring chunk sits at the end of the context block. The claim is that 8B-class recency

dominance, established by Liu et al. at the nearby 7B scale, makes the final position the single most attended slot and therefore the right place for the highest-scoring chunk.

Primacy–Recency placement places chunks alternately at the end and the start of the context block, walking inward from both bookends. Rank 1 lands at the end (the recency anchor), rank 2 lands at the start (the primacy anchor), rank 3 lands one slot before rank 1, rank 4 lands one slot after rank 2, and so on, alternating until all admitted chunks are placed. Hence, the two highest-ranked chunks anchor the two ends and the lowest-ranked admitted chunk lands in the very middle of the prompt, where the attention intensity is the lowest. The strategy collapses to recency-biased for $N \leq 2$, so very short admitted sets behave identically under both strategies.

4.8 The GPU Shuttling Mechanism

The pipeline is built for a single consumer laptop: an RTX 3060 Laptop GPU with 6 GB of VRAM, paired with 16 GB of system RAM. The pipeline runs four GPU-resident models: Qwen3-4B embedder, the **bge-reranker-large** cross-encoder, the Llama 3.1 generator, and the **DeBERTa-v3-large** NLI faithfulness checker whose summed peak VRAM occupation is approximately 10 GB if all are resident simultaneously. Therefore, all four cannot be in VRAM at the same time.

The architectural response is the GPU shuttling pattern: each model is loaded into VRAM only when its layer is active and is moved back to CPU before the next layer’s model is loaded. However, this shuttling comes with the cost of extra latency overhead on every layer transition where the GPU shuttling occurs. The cost is real, but it is the price the pipeline pays to run at all on the available hardware.

The shuttling pattern is the architectural answer to Research Question 4, and the answer is not one that runs faster than the cloud and is not one that matches frontier-model quality, but one that gives honest confidence signals while responding, and pays predictable latency cost for full local deployability.

5. CHAPTER: ABLATION SETUP

Twelve variants are run, organized into three realms. The Vanilla realm is the ultimate floor that has dense-only retrieval and a naive system prompt. The Recency-Biased realm uses the full discriminative pipeline with linear descending context placement (worst chunk first, best chunk last). The Primacy-Recency-Biased realm uses the same discriminative pipeline with the alternating placement strategy. The full ablation variant list is given in Table 5.1.

Table 5.1: Ablation Variants

Realm	Variant	What it isolates
Vanilla	Top-5	Floor: pipeline vs naive at k=5
Vanilla	Top-10	Floor: pipeline vs naive at k=10, controls for context size
Recency-Biased	Dynamic Top-k (gate threshold)	Gated admission with linear placement
Recency-Biased	Top-5	Fixed-k linear placement, smaller context
Recency-Biased	Top-10	Fixed-k linear placement, larger context
Recency-Biased	No-Reranker Top-5	Reranker contribution at k=5 under recency placement
Recency-Biased	No-Reranker Top-10	Reranker contribution at k=10 under recency placement

Table 5.1 (continued): Ablation Variants

Primacy-Recency	Dynamic Top-k (gate threshold)	Gated admission with alternating placement
Primacy-Recency	Top-5	Fixed-k alternating, smaller context
Primacy-Recency	Top-10	Fixed-k alternating, larger context
Primacy-Recency	No-Reranker Top-5	Reranker contribution at k=5 under alternating placement
Primacy-Recency	No-Reranker Top-10	Reranker contribution at k=10 under alternating placement

The Vanilla pipeline strips every architectural addition that Chapter 4 layers onto plain RAG. Retrieval is dense only: the **Qwen3-Embedding-4B** encoder over the same unified ChromaDB collection, with no domain filter, no BM25, and no rank fusion. There is no cross-encoder reranker and therefore no gating or admission decision. The system prompt is a canonical naive-RAG template (“use the following pieces of context; if you do not know, say so”) with none of the cross-domain interaction rules, format guides, or attribution conventions described in section 4.4. Context placement is dense-rank best-first (the opposite of recency-biased placement), and the query is not placed after the context block. Generation hyperparameters (temperature, top-p, max tokens, seed) match the discriminative variants exactly, so any cross-realm quality difference is attributable to architecture rather than to sampling. (See Appendix 2 for hyperparameter values.)

The five variants in each realm of the full pipeline differ only in the Layer 2 admission rule and (across realms) in the context-placement strategy. Dynamic Top-k uses the cross-encoder gate at threshold 0.45 (the baseline behavior), with the admitted count varying per query and Option-C fallback admitting a single chunk when no candidate clears the threshold. Fixed Top-k ($k = 5$ or 10) runs the same cross-encoder scoring but admits a constant number of top-scoring chunks, with the gate bypassed entirely. No-Reranker Top-k ($k = 5$ or 10) replaces the cross-encoder with a passthrough that admits the first k chunks of the global RRF order.

6. CHAPTER: EVALUATION METHODOLOGY

6.1 Benchmark Design and Construction

Evaluating a cross-domain RAG pipeline requires a benchmark aligned with the specific failure modes the system is intended to address. A single homogeneous query set would be insufficient for this purpose. Therefore, the evaluation benchmark is divided into three query tiers: (1) queries with clear factual answers, (2) queries whose ambiguity may induce cross-domain confusion, and (3) queries whose premises are unsatisfiable. Each tier is constructed and evaluated using a methodology appropriate to its characteristics.

6.1.1 Tier 1: Clean Factual

Tier 1 (T1) consists of factual questions with a single correct answer grounded in specific chunks. Each T1 question is associated with a reference answer and with up to four ground-truth chunk identifiers. The reference answers and ground-truth chunk lists enable both retrieval-level metrics and reference-grounded answer-level metrics for this tier. T1 is the only tier with this structure; T2 and T3 are evaluated by rubric.

6.1.2 Tier 2: Cross-Domain Attribution Under Ambiguity

Tier 2 (T2) consists of queries that use vague or general terminology where retrieval can plausibly utilize chunks from both domains. The answer space admits multiple valid framings: presenting both domains with explicit per-domain attribution or presenting one domain correctly with attribution when retrieval for the other is insufficient. The failure mode the tier tests for is attribution discipline: mixing mechanics across domains without distinguishing them or omitting attribution entirely and leaving the reader to guess which game the answer concerns. A single authored reference answer does not fit T2 because more than one valid answer is admissible, which is why this tier is evaluated by rubric.

6.1.3 Tier 3: Abstention Under Invalid Premise

Tier 3 (T3) consists of queries whose premise cannot be satisfied. T3 questions fall into two construction patterns: gibberish or made-up terms that do not exist in either domain, or queries that combine real entities across domains to form connections that cannot exist. The expected behavior is abstention. The system should recognize that the query is ungrounded or rely on an invalid premise and should decline to construct an answer. Correct in-domain information with correct per-game attribution is accepted as a fallback provided that no cross-domain reasoning is constructed. Like T2, T3 is also evaluated by a rubric; a reference-grounded scorer cannot distinguish a correct abstention from a vacuous one.

The query counts per tier are shown in Table 6.1.

Table 6.1: Query Amount per Tier

Tier	Questions
T1: Clean Factual	194
T2: Cross-Domain Attribution under Ambiguity	72
T3: Abstention under Invalid Premise	57
Total	323

6.2 Evaluation Framework

The evaluation framework follows the benchmark’s stratification: each tier is scored by a method that is fitted to its nature. No single metric group covers all three tiers. Imposing a uniform metric would either underscore T2 and/or T3 (because a reference-based scorer cannot capture multi-framing answers or correct abstention) or overscore them (because a scorer would treat any plausible-sounding output as acceptable). The three-regime design is the response to that.

Tier 1 is evaluated by RAGAS with retrieval-level Hit@k and MRR computed alongside; Tier 2 is evaluated by an attribution-and-correctness rubric; Tier 3 is

evaluated by an abstention rubric. System-level retrieval metrics are restricted to T1 because they require one positive ground truth.

6.3 Tier-1 Evaluation

6.3.1 RAGAS Metric Stack

Five RAGAS metrics are computed per T1 question. Two of them (Context Recall, Answer Correctness) require the authored reference answer. The five together provide a multidimensional picture of T1 quality: retrieval is captured by Context Recall and Context Precision, generation grounding is captured by Faithfulness, and end-to-end answer quality is measured by the pair Answer Correctness (vs reference) and Answer Relevancy (vs question). A summary of the mentioned metrics is shown in Table 6.2.

All RAGAS scoring is performed by Qwen3.5-72B, served through an OpenAI-compatible vLLM endpoint running on Google Colab.

Table 6.2: RAGAS Metrics

Metric	What it measures	Reference needed?
Faithfulness	Claim-level entailment of the generated answer from the retrieved context	No
Context Recall	Proportion of reference-answer content recoverable from the retrieved context	Yes
Context Precision	Proportion of retrieved context that is relevant to the reference answer	No
Answer Correctness	Semantic similarity of the generated answer to the authored reference answer	Yes
Answer Relevancy	Alignment of the generated answer with the question, judged independently of the reference	No

6.3.2 Manual Oversight

RAGAS scoring is complemented by an exhaustive manual scoring pass over Tier 1: every T1 answer, across every variant, is read against its authored reference and marked correct or incorrect by a human grader. The manual pass is a parallel score, not a sample audit. Its purpose is to provide a reference-anchored binary judgement that is independent of the RAGAS judge’s semantic-similarity scoring, so that Chapter 7 can report both signals side by side and the reader can see where they agree and where they diverge: semantic similarity and reference correctness are related but distinct, and mixing them would hide rather than reveal the whole picture.

6.3.3 Retrieval Metrics

Retrieval quality on T1 is measured by two classical information retrieval (IR) set-based metrics computed against the ground-truth chunk lists. Hit@k reports, for each query, whether at least one chunk from `source_chunk_id_1 ... source_chunk_id_4` appears among the top-k retrieved candidates from Layer 1. MRR (Mean Reciprocal Rank) reports the reciprocal of the rank at which the first ground-truth chunk appears, averaged across queries. Both are computed as set operations on chunk identifiers, agnostic to chunk content and answer text. These metrics are restricted to T1 because they require positive ground truth: T2 admits multiple valid retrievals, and T3 has no positive ground truth at all.

6.4 Tier-2 Evaluation

Tier 2 is evaluated by rubric rather than by reference as mentioned in the previous sections. It scores both attribution and correctness jointly: an answer scores Pass only if it addresses the query, respects attribution discipline, and contradicts nothing in the retrieved context.

6.4.1 Rubric

The three dimensions of the rubric are summarized in Table 6.3. The full version can be found in Appendix 4.

Table 6.3: Tier-2 Rubric

Dimension	Question	Options
Q1: Addresses query	Does the answer address the query?	Yes / Partial / No
Q2: Attribution scope	Is the answer scoped to a specific game or games?	Both-attributed / One-attributed / Mixes-mechanics
Q3: Contradiction	Does any claim contradict the retrieved context?	No / Yes

6.4.2 Scoring

Each T2 question is awarded one of three scores derived from the rubric responses (Table 6.4).

Table 6.4: Tier-2 Rubric Scoring

Score	Label	Conditions
2	Pass	Q1 Yes; Q2 $\in$ {both-attributed, one-attributed (if it only contains information from that domain)}; Q3 No
1	Partial	Q1 Partial; Q2 {both-attributed, one-attributed (if it only contains information from that domain)}; Q3 No
0	Fail	Q1 NO, OR Q2 mixes-mechanics, OR Q3 Yes

The T2 score is the proportion of the maximum achievable score across all 72 questions (144 points). Scoring is applied by a single human grader; in cases of doubt, the rubric is applied conservatively (a borderline answer is scored Partial rather than Pass, a borderline cross-domain claim is scored Fail rather than Partial).

6.4.3 Evaluation Example

Below is the example Tier-2 query evaluation for partially correct case. Additional Pass and Fail examples are given in Appendix 4.

Table 6.5: Partially Correct Example

Query	What is the maximum number of wards a single player can place on the map at one time?
Answer	In League of Legends, a single player can have a maximum of three green wards on the map at one time.
Ground Truth (LoL Variant)	A player can place up to 3 Stealth Wards and 1 Control Ward at the same time.
Evaluation	Q1 - Partial (answers a narrower question, covers only one ward type); Q2 - one-game with explicit attribution; Q3 - not in contradiction with the context retrieved but the Control Ward information is missing. Hence, the score = 1.

6.5 Tier-3 Evaluation

Tier 3 evaluates whether the system recognizes and refuses queries whose premise cannot be satisfied. The rubric has two options to grade an answer: it is either a correct handling (Pass, 1 point) or an invalid premise violation (Fail, 0 points). The rubric is presented in Table 6.6.

Table 6.6: Tier-3 Rubric

Score	Label	Criteria
1	Pass	The answer does at least one of: (a) explicitly states the entities belong to different games or that the premise is invalid; (b) refuses to answer on the grounds that the

		context does not support it; (c) describes each entity separately with per-game attribution, making the cross-game mismatch evident; (d) answers correctly for one entity with clear attribution and does not fabricate or speculate about the other.
0	Fail	The answer constructs a mechanical interaction, effect, comparison, or outcome between the entities. This includes unhedged fabrication (“X counters Y”), hedged speculation (“we can infer that X would affect Y”), and conditional or hypothetical framing (“if they interacted, X would ...”). Any construction of cross-domain reasoning fails, regardless of the epistemic markers used to frame it.

The crucial design decision is that epistemic hedging does not rescue a cross-domain claim. The Fail criterion explicitly covers hedged speculation and conditional or hypothetical framing because the failure mode T3 tests for is the construction of cross-domain reasoning yielding impossible interaction, not the confidence with which it is delivered. An answer that wraps a fabricated interaction such as “we can infer that...” or “if they interacted...” is still constructing the interaction.

The T3 score is the proportion of Pass scores across all 57 questions. Application protocol mirrors T2: a single grader applies the rubric, with conservative scoring: a borderline hedged construction is scored Fail rather than Pass.

6.5.1 Evaluation Example

Below is the example Tier-3 query evaluation for “Pass” case. Additional examples are given in Appendix 4.

Table 6.7: Fully Correct Example

Query	How much bonus armor does the Vanguard shield provide to Baron Nashor when purchased from the side shop?
Answer	I cannot provide information about interactions between entities from different games. The Vanguard shield is from the game Dota 2, while Baron Nashor is from the game League of Legends.

Table 6.7 (continued): Fully Correct Example

Evaluation	The answer recognizes the invalid premise, names both entities with explicit per-game attribution, and constructs no interaction. Score = 1
------------	---

6.6 Inference on Scores

Because manual evaluation results in binary outcomes (success or failure) per query, each variant's success rate on a tier is a sample proportion drawn from a finite question set. To distinguish substantive performance differences from noise, this work computes the standard error of the difference between two proportions for every reported pairwise comparison:

$$SE_diff = \sqrt{\frac{p_1 \cdot (1 - p_1)}{N} + \frac{p_2 \cdot (1 - p_2)}{N}} \quad (2)$$

where p_1 and p_2 are the observed success rates of the two variants under comparison and N is the tier size. The SE_diff value, multiplied by N , expresses the expected sampling fluctuation in question counts. An observed gap between two variants is then characterized by its ratio to SE_diff : gaps of one standard error (1σ) carry approximately 32% probability of arising from sampling noise alone under the null hypothesis of no true difference; two standard errors (2σ) correspond to approximately 5% probability; three standard errors (3σ) to approximately 0.3%. Following conventional academic practice, the 2σ threshold is treated as the level of reportable findings, 1σ – 2σ as defensible observations, and below 1σ as within-noise observations reported only as percentage-point differences without claim to statistical distinguishability.

One important caveat exists that the independent-proportions formula assumes the two samples are unrelated, while in this evaluation every variant runs on the same question set. The shared-question structure positively correlates the estimates (a hard question tends to fail across variants, an easy question tends to succeed across variants), so the true SE of the difference is smaller than the independent-proportions formula returns

since the p-value would be lower than that of reported ones. Hence, the reported thresholds are conservative.

7. CHAPTER: RESULTS AND DISCUSSION

7.1 Headline Results

The results show at every tier that Vanilla Top-5 and Top-10 are clearly separated from the full pipeline on both T1 and T2; the pipeline contributes a considerable reduction in hallucination problem on cross-domain QA. Within the ten full-pipeline variants, the spread on T1 is comparatively narrow (87.1%-95.4%), while T2 and T3 show larger inter-variant separation.

The best-performing T1 variant (Primacy-Recency (PR)-Dynamic, 95.4%) also ranks first on T3 (94.7%), and PR-Reranker-Top-5 is the best-performing T2 variant at 75.0%. The two dynamic variants and PR-Reranker-Top-5 place at the top of every tier ranking, but the upcoming sections show that the gaps between full-pipeline variants are tighter than the headline numbers and that several rankings sit within the noise floor on T2 and T3.

Vanilla variants are not evaluated on Tier-3 since Vanilla variants produce “I don’t know” responses on 68 (Top-10) and 79 (Top-5) non-T3 queries anyways, and even when it starts the answer with abstention wording, it often continues by providing an answer afterwards. Therefore, there is no clean separation between real abstention and surface-level hedging for Vanilla variants in T3 queries.

Table 7.1 summarizes every variant’s headline manual success rate per tier.

Table 7.1: Success Rates per Tier per Variant

Variant	Tier-1	Tier-2	Tier-3
Vanilla Top-5	64.9%	29.2%	-
Vanilla Top-10	68.0%	44.4%	-

Table 7.1 (continued): Success Rates per Tier per Variant

Recency- NoReranker-Top-5	87.6%	63.2%	82.5%
Recency- NoReranker-Top-10	90.7%	69.4%	87.7%
Recency-Reranker- Top-5	93.3%	68.1%	80.7%
Recency-Reranker- Top-10	92.3%	69.4%	87.7%
Recency-Dynamic	94.8%	72.2%	93.0%
PR-NoReranker- Top-5	87.1%	65.3%	86.0%
PR-NoReranker- Top-10	91.8%	70.8%	86.0%
PR-Reranker-Top-5	93.8%	75.0%	91.2%
PR-Reranker-Top-10	93.3%	68.8%	86.0%
PR-Dynamic	95.4%	72.9%	94.7%

Table 7.2 reports the RAGAS scores on T1 for each variant in median (mean) form. Hit@k and MRR are reported as means.

Table 7.2: RAGAS Scores of Tier-1. (in median (mean) format)

Variant	Faith.	Recall	Precis.	Correct.	Relev.	Hit@1	Hit@5	MRR
Vanilla Top-5	1.00 (0.74)	1.00 (0.70)	0.78 (0.60)	0.64 (0.60)	0.89 (0.64)	0.47	0.68	0.55

Table 7.2 (continued): RAGAS Scores of Tier-1 (in median (mean) format)

Vanilla Top-10	1.00 (0.78)	1.00 (0.76)	0.61 (0.58)	0.73 (0.65)	0.89 (0.67)	0.47	0.68	0.56
Recency- NoReranker- Top-5	1.00 (0.91)	1.00 (0.88)	0.83 (0.68)	0.82 (0.75)	0.93 (0.88)	0.49	0.85	0.63
Recency- NoReranker- Top-10	1.00 (0.92)	1.00 (0.96)	0.70 (0.66)	0.82 (0.77)	0.93 (0.90)	0.50	0.85	0.63
Recency- Reranker- Top-5	1.00 (0.93)	1.00 (0.96)	1.00 (0.94)	0.83 (0.79)	0.93 (0.88)	0.81	0.93	0.87
Recency- Reranker- Top-10	1.00 (0.92)	1.00 (0.96)	1.00 (0.94)	0.83 (0.77)	0.93 (0.89)	0.81	0.93	0.87
Recency- Dynamic	1.00 (0.92)	1.00 (0.96)	1.00 (0.95)	0.83 (0.78)	0.94 (0.90)	0.81	0.93	0.87
PR- NoReranker- Top-5	1.00 (0.89)	1.00 (0.88)	0.95 (0.69)	0.77 (0.74)	0.93 (0.88)	0.50	0.85	0.63
PR- NoReranker- Top-10	1.00 (0.94)	1.00 (0.96)	0.71 (0.67)	0.83 (0.79)	0.93 (0.89)	0.50	0.85	0.63
PR- Reranker- Top-5	1.00 (0.91)	1.00 (0.96)	1.00 (0.94)	0.83 (0.78)	0.93 (0.90)	0.81	0.93	0.87
PR- Reranker- Top-10	1.00 (0.92)	1.00 (0.96)	1.00 (0.94)	0.82 (0.78)	0.93 (0.89)	0.81	0.93	0.87
PR- Dynamic	1.00 (0.92)	1.00 (0.96)	1.00 (0.95)	0.83 (0.78)	0.94 (0.90)	0.81	0.93	0.87

7.2 Reranker Contribution

The reranker contributes substantially to Tier-1 performance, particularly at fixed $k=5$ configurations. Paired comparisons of Reranker-Top-5 against No-Reranker-Top-5 variants sit in the vicinity of the 2 standard errors (2σ) which corresponds to $\sim 95\%$ two-sided confidence for standard normal distribution, on T1 scores. PR-Reranker-Top-5 versus PR-NoReranker-Top-5 is a 6.7 ppt (13-question) gap at 2.26σ ($\sim 97.6\%$ two-sided confidence). Similarly, Recency-Reranker-Top-5 versus Recency-NoReranker-Top-5 lands at 1.91σ , ($\sim 94.4\%$ confidence) just below 2σ , but consistent in direction and magnitude.

On T2, the reranker’s contribution narrows. PR-Reranker-Top-5 versus PR-NoReranker-Top-5 is 9.7 ppt, however that difference sits at only 1.28σ ($\sim 80.0\%$) on $N=72$. The matched-placement Reranker-versus-NoReranker pairs at other configurations sit at 4–6 ppt and below 1.5σ . The interpretation is that T2 success is driven more by output-side attribution than by retrieval discrimination at the chunk level. It is understood that the reranker can sharpen the admitted set but does not rescue cross-domain attribution by itself.

On T3, the reranker has no eye-catching independent effect. At Recency-Top-5 the reranker-versus-no-reranker delta is 1.76 ppt; at Top-10 across both placements the differences are within 0-1.76 ppt. Removing the reranker therefore does not break abstention behavior on its own.

A complementary observation from the RAGAS scores on T1 reinforces the reranker’s contribution. Median context precision reaches 1.00 only in reranker-equipped variants (including Dynamic); No-Reranker variants drop to median 0.61-0.95 and even lower in the mean scale.

7.3 System Prompt Contribution

The system prompt carries most of the T2 contribution, while the retrieval architecture supplies a smaller and noisier secondary benefit. A recurring failure mode in the Vanilla variants was not factual inaccuracy itself, but missing domain attribution. Many T2 answers contained gameplay information that was individually correct, yet failed the evaluation because the response did not explicitly specify whether the information referred to Dota 2 or League of Legends. In a cross-domain setting, such

answers remain ambiguous to the end user, since both games may plausibly claim similar mechanics or terminology.

To isolate this effect, an additional conditional record was maintained during manual evaluation. Vanilla outputs were re-examined under an “if-it-references” condition: cases where the answer would have been considered correct if the model had explicitly identified the target game. Under this conditional baseline, the attribution rate rises to 60.4% for Vanilla Top-5 and 69.4% for Vanilla Top-10. This indicates that a substantial portion of T2 errors originates from missing attribution rather than failed retrieval or fabricated content.

The full-pipeline system prompt explicitly enforces domain attribution in generated answers. Comparing PR-Reranker-Top-5 (75.0%) against the Vanilla Top-10 conditional baseline (69.4%) yields a residual improvement of 5.6 ppt at approximately 0.75σ (~54% confidence) on $N=72$, placing the architecture’s incremental T2 contribution within sampling noise. Therefore, the evidence suggests that the dominant T2 improvement comes from the system prompt’s attribution enforcement, while the retrieval pipeline provides a directional but statistically inconclusive secondary benefit.

7.4 Context Positioning

Primacy-Recency-biased placement (PR) and Recency-biased placement (R) were compared at matched k and reranker presence. On T1, placement effects are not detectable. All four matched PR-versus-R comparisons at fixed reranker presence and k are within 0–1.0 ppt; none separates from sampling noise. PR-Dynamic versus Recency-Dynamic is 0.5 ppt; PR-Reranker-Top-5 versus Recency-Reranker-Top-5 is 0.5 ppt; PR-NoReranker-Top-10 versus Recency-NoReranker-Top-10 is 1.0 ppt. Hence, it can be said that the pipeline saturates on clean factual queries to the point where positional attention to the golden chunk is rarely the bottleneck.

On T2, PR-Reranker-Top-5 is the headline-best variant at 75.0%, but the placement effect at matched configuration is not above noise. PR-Reranker-Top-5 versus Recency-Reranker-Top-5 is 6.9 ppt observed at only 0.93σ (~64.8% confidence). The sweet spot is real as a top observed value, but the placement effect does not separate from sampling noise at $N=72$; reading too strong conclusion from the matched comparison would over-interpret the data.

On T3, placement at fixed reranker presence and varying k shows the most suggestive directional effect. PR-Reranker-Top-5 versus Recency-Reranker-Top-5 is 10.5 ppt at 1.64σ (~89.9% confidence) defensible as directional but below the 2σ threshold (95%). Substituting PR placement improves abstention to 91.2%, and this improvement is the placement effect rather than a reranker or k effect because both are held fixed.

7.5 Dynamic Gating vs Fixed Top-k

Dynamic Top-k adds threshold-based admission of chunks on top of the cross-encoder scoring used by Fixed Top-k variants. Pure gating is within noise on T1, but it is consistent in leading rankings: PR-Dynamic versus PR-Reranker-Top-5 is 1.6 ppt observed, and Dynamic versus Reranker-Top-10 across both placements sits at 1.45σ – 1.80σ (~85–93% confidence). The interpretation is that the reranker carries most of the T1 contribution, and the gate provides directional improvement that does not separate cleanly from the reranker at $N=194$.

PR-Dynamic versus Recency-Dynamic on T3 is 1.76 ppt (one question), statistically indistinguishable. Dynamic’s benefit on abstention is therefore placement-agnostic. This makes the gate threshold as the operative mechanism on T3. LLM always sees k chunks regardless of their relevance at fixed k , so it must reason out an abstention from low-quality evidence. On the contrary, with Dynamic, weak-evidence chunks are filtered before the LLM ever sees them, and the LLM’s default conservative behavior produces the abstention, and once the chunks are filtered, the order they would have been placed in becomes irrelevant.

7.6 Hardware Feasibility and Latency

The median latency (in seconds) of variants is shown in Table 7.3. Full latency table is available in Appendix 5.

Table 7.3: Median Latency per Variant (in Seconds)

Variant	Pipeline-Total median (s)	GPU-shuttle median (s)	Net Latency (s)
Vanilla Top-5	17.4	-	17.4

Table 7.3 (continued): Median Latency per Variant (in Seconds)

Vanilla Top-10	17.7	-	17.7
Recency- NoReranker-Top-5	30.8	12.5	18.3
PR-NoReranker- Top-5	32.8	12.5	20.3
PR-NoReranker- Top-10	36.8	12.9	23.9
Recency-Dynamic	37.1	13.4	23.7
Recency- NoReranker-Top-10	40.0	13.2	26.8
PR-Dynamic	40.4	15.4	25.0
Recency-Reranker- Top-5	41.6	13.6	28.0
PR-Reranker-Top-5	43.3	14.6	28.7
Recency-Reranker- Top-10	52.5	14.9	37.6
PR-Reranker-Top-10	55.7	15.2	40.5

Several observations follow. First, the GPU shuttle is a flat ~12–15 s overhead across every full-pipeline variant. This is the cost the architecture pays for running on a 6 GB-

VRAM consumer laptop: the four GPU-resident models cannot co-exist, so each must be loaded and unloaded per query. The shuttle is approximately 30-40% of full-pipeline latency for the lighter variants and 25-30% for the heaviest. Second, the Vanilla variants at ~17.5 s do not include reranker or faithfulness-checker loads and therefore set a hard lower bound on per-query latency at ~17 s. Third, Dynamic variants sit in the middle of the range; they provide confidently-retrieved chunks but still pay the full shuttle overhead.

The results therefore show that the pipeline remains feasible on consumer-grade hardware, but latency becomes dominated by model-shuttling overhead rather than retrieval computation itself.

7.7 Discussion

Mechanism-aware interpretation matters more than headline ranking. PR-Dynamic ranks first on T1 (95.4%) and T3 (94.7%); PR-Reranker-Top-5 ranks first on T2 (75.0%). At first glance, these values suggest per-tier deployment recommendations. However, the deeper picture from Sections 7.2-7.5 is that different tiers are dominated by different mechanisms: T1 by the reranker, T3 by the gate threshold, T2 by the system prompt with the retrieval stack contributing to a secondary residual. The strongest finding the benchmark supports is therefore not “PR-Dynamic is the best variant overall” but “the components that earn the headline numbers are not the same across tiers, and a smaller architecture that dropped any one of the three would lose a different part of the picture.” The headline number is a compound effect; the contributions decompose differently in each setting.

RAGAS as complementary, not authoritative. RAGAS scoring was used on T1 alongside exhaustive manual scoring. The two signals agree on the rough variant ordering but diverge in absolute calibration. Median faithfulness and context recall saturate at 1.00 across all variants, so the median T1 question is solved everywhere, including even by Vanilla, but the architectural contribution shows in the means (faithfulness mean ~0.74-78 for Vanilla, ~0.91-94 for full pipeline). Median context precision reaches 1.00 only for reranker-equipped variants. Answer correctness, by contrast, is systematically lower in RAGAS than in manual scoring across full-pipeline variants (medians mostly around 0.82-0.83 against manual rates around 0.93-0.95). This is not a calibration failure to correct; it is the property of what RAGAS answer

correctness measures: semantic similarity to a reference answer which is related to but distinct from factual correctness. A correct answer that adds substantive detail not in the reference scores lower; an incorrect answer phrased similarly to the reference scores higher. The treatment in this work is to report both side by side and to pick manual scoring as authoritative, not to discard RAGAS, which provides the precision and recall information that manual binary scoring does not.

8. CHAPTER: CONCLUSION

This work addressed cross-domain question answering in settings where two corpora share substantial surface vocabulary but differ in underlying mechanics; the mixture-of-contents problem. The proposed solution is a discriminative pipeline composed of hybrid retrieval, gated cross-encoder reranking, generation, NLI-based faithfulness checking, and confidence orchestration, engineered to run on a 6 GB-VRAM consumer laptop. Two empirical artefacts were produced: a 323-question stratified benchmark covering clean factual queries (T1: 194), ambiguous cross-domain queries (T2: 72), and abstention-required invalid-premise queries (T3: 57); and a twelve-variant ablation study isolating the contribution of each architectural component.

Across the four research questions, the pipeline produced four substantive findings: the reranker contributes approximately 6 percentage points of T1 accuracy, exceeding 2σ in the primacy-recency placement (and just below in the recency placement); the gate threshold drives T3 abstention with PR-Dynamic; positioning effects are directional but below 2σ across tiers on the present sample sizes; and the cross-domain attribution gap on T2 decomposes into a dominant system-prompt contribution ($\sim 20\text{--}30$ ppt) and a smaller architecture residual (3-6 ppt at noise boundary).

Methodologically, the work reveals cross-domain terminological collision as an under-examined failure mode and offers a tier-stratified evaluation lens that separates clean factual answering, cross-domain attribution discipline, and abstention under invalid premise into distinct testable behaviors. The directions that follow from the limitations of the present setup are taken up in Chapter 9.

9. CHAPTER: FUTURE WORK

9.1 Limitations

Several limitations bound the scope of the claims this report can make. The work is built and evaluated on a single corpus pair (Dota 2 and League of Legends) in a single language (English), and a single LLM family generating answers. The 323-question benchmark is large enough to surface architectural differences across the twelve ablation variants but small in absolute terms, particularly for the rubric-graded tiers (T2: 72; T3: 57).

Three further limitations are architectural rather than empirical. The pipeline was built for a 6 GB-VRAM consumer laptop, and that constraint shaped almost every model choice; the work demonstrates what is feasible under that constraint, not that the chosen configuration is optimal. The pipeline is also deliberately a frozen-models architecture, which means any quality gain is accessible only through fine-tuning. A third is the lack of multi-hop and active retrieval, in which a second retrieval round is triggered conditional on the first or on-demand mid-generation.

9.2 Methodological and Evaluation Extensions

Each chunk carries a structured metadata schema of which only the game field is currently consumed at retrieval time; metadata-aware retrieval that routes entity-named queries is a natural next step. T2 rubric scoring could be partially automated by a classifier or rule-based detector trained on the explicit per-game attribution patterns in correct T2 answers. A parametric-override study following Mallen et al. (2023) characterizing when the LLM answers from parametric memory versus from retrieved context would identify where the discriminative scaffolding is most needed in the cross-domain ambiguous regime.

10. REFERENCES

- Amiraz, C., Cuconasu, F., Filice, S., & Karnin, Z. (2025). The distracting effect: Understanding irrelevant passages in RAG. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 18228–18258.
- Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *The Twelfth International Conference on Learning Representations*.
- Chirkova, N., Rau, D., Déjean, H., Formal, T., Clinchant, S., & Nikoulina, V. (2024). Retrieval-augmented generation in multilingual settings. *Proceedings of the 1st Workshop on Towards Knowledgeable Language Models (KnowLLM 2024)*, 177–188.
- Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 758–759. <https://doi.org/10.1145/1571941.1572114>
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 150–158. <https://doi.org/10.18653/v1/2024.eacl-demo.16>
- Falke, T., Ribeiro, L. F. R., Utama, P. A., Dagan, I., & Gurevych, I. (2019). Ranking generated summaries by correctness: An interesting but challenging application for natural language inference. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2214–2220. <https://doi.org/10.18653/v1/P19-1213>
- Hoffart, J., Yosef, M. A., Bordino, I., Fürstenau, H., Pinkal, M., Spaniol, M., Taneva, B., Thater, S., & Weikum, G. (2011). Robust disambiguation of named entities in text. *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, 782–792.
- Hsieh, C.-Y., Chuang, Y.-S., Li, C.-L., Wang, Z., Le, L., Kumar, A., Glass, J., Ratner, A., Lee, C.-Y., Krishna, R., & Pfister, T. (2024). Found in the middle: Calibrating positional attention bias improves long context utilization. *Findings of the*

- Association for Computational Linguistics: ACL 2024, 14982–14995. <https://doi.org/10.18653/v1/2024.findings-acl.890>
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), Article 248. <https://doi.org/10.1145/3571730>
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- Kryściński, W., McCann, B., Xiong, C., & Socher, R. (2020). Evaluating the factual consistency of abstractive text summarization. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 9332–9346. <https://doi.org/10.18653/v1/2020.emnlp-main.750>
- Laurer, M., van Atteveldt, W., Casas, A., & Welbers, K. (2023). Building efficient universal classifiers with natural language inference. *arXiv*. <https://doi.org/10.48550/arXiv.2312.17543>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173. https://doi.org/10.1162/tacl_a_00638
- Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., & Hajishirzi, H. (2023). When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 9802–9822.
- Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On faithfulness and factuality in abstractive summarization. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 1906–1919. <https://doi.org/10.18653/v1/2020.acl-main.173>
- Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective retrieval augmented generation. *arXiv*. <https://doi.org/10.48550/arXiv.2401.15884>

APPENDICES

APPENDIX 1: MODEL SPECIFICATIONS

Six models are referenced in the report. The first four are GPU-resident at inference time and constitute the discriminative pipeline. The fifth is an off-device judge model used for RAGAS evaluation. The sixth (**jina-reranker-v3**) is referenced in Chapter 4 as a calibration alternative considered and not adopted; its specifications are listed for completeness. All license and access metadata were verified on 18 May 2026 via the model cards on Hugging Face.

Table A1.1: Model Specifications

Model	Model Identifier	Quant.	Size	Context Length	License
Qwen3-Embedding-4B	Casual-Autopsy/Qwen3-Embedding-4B-GGUFs	Q8_0	~4.2 GB / ~4.0 GB	32768	Apache 2.0
bge-reranker-large	BAAI/bge-reranker-large	FP16	~1.3 GB / ~1.2 GB	512	MIT
Llama 3.1 8B	ollama.com/library/llama3.1:8b	Q4_K_M	~4.9 GB / ~5.0 GB	128000	Llama 3.1
DeBERTa-v3-large-zeroshot-v2.0	MoritzLaurer/deberta-v3-large-zeroshot-v2.0	FP16	~870 MB / ~900 MB	512	MIT
Qwen3.5 27B (judge)	Qwen/Qwen3.5-27B	FP16	~55.4 GB	262144	Apache 2.0

Table A1.1 (continued): Model Specifications

jina-reranker-v3	jinaai/jina-reranker-v3	FP16	~1.2 GB / not deployed	131072	CC BY-NC 4.0
------------------	-------------------------	------	------------------------------	--------	--------------

Two clarifying notes. First, the Llama 3.1 8B Q4_K_M weights were sourced through Ollama which packages the same Llama-3.1-8B base under the Llama 3.1 Community Licence; the underlying GGUF blob is consumed directly by llama-cpp-python at inference.

APPENDIX 2: HYPERPARAMETERS AND CONFIGURATION

This appendix consolidates the operating parameters of the discriminative pipeline. All values reflect the configuration used to generate the Chapter 7 results. The full source code, including these defaults, is available at <https://github.com/c-ozd/term-project>.

Table A2.1: Reranker Configuration

Configuration	Value	Notes
Gate threshold	0.45	Unipolar Sigmoid-normalized, on cross-encoder score
Sigmoid temperature	$T = 4.0$	Normalization temperature
Reranker max pair length	512 tokens	Internal <code>_SAFE_MAX_LENGTH</code>
Max query tokens (gate bypass)	150 tokens	<code>_MAX_QUERY_TOKENS</code>
Assembled context budget	3500 tokens	<code>max_context_tokens</code>
Option-C fallback	1 chunk	When no candidate clears the gate

Table A2.2: Aggregated Score Weights and Answer Tier Bands

Parameter	Value	Notes
Weight - reranker median	$w_R = 0.70$	-
Weight - faithfulness max	$w_F = 0.30$	-
Tier - HIGH	> 0.70	Answer + citations
Tier - MODERATE	> 0.50	Answer + citations + uncertainty note
Tier - LOW	≥ 0.20	Answer + citations + low-confidence warning (inclusive)
Tier - DEAD	< 0.20	Abstain; ask user to rephrase

Table A2.3: LLM Decoding Parameters

Parameter	Value	Notes
Temperature	0.5	Moderate sampling
Top-p	0.9	Nucleus sampling cutoff
Max generated tokens	512	Generation cap
Context window (n_ctx)	5000	Fits 3500-token context + headroom
Random seed	42	Reproducibility

Table A2.4: Corpus Chunking and Retrieval Parameters

Parameter	Value	Notes
Chunk size budget	512 tokens	Inclusive of structured-header prefix
Chunk overlap	50 tokens	Recursive character splitter
RRF constant	k = 60	Cormack et al. (2009)
Dense retriever n_ctx	8192	-
Dense retriever n_batch	4096	-

APPENDIX 3: SYSTEM PROMPTS

Vanilla System Prompt

SYSTEM:

You are a helpful assistant for question-answering tasks. Use the following pieces of retrieved context to answer the user's question. If the context does not contain the information needed, say that you do not know. Be concise.

Pipeline System Prompt

SYSTEM:

You are a knowledgeable assistant for competitive gaming wikis covering two separate games: Dota 2 and League of Legends. Answer the user's question using ONLY the context provided below.

CROSS-DOMAIN INTERACTION RULE (critical):

- Each chunk in the context begins with a structured header that identifies its game, for example 'Game: Dota 2 | Hero: ... | Data: ...' or 'Game: League of Legends | Champion: ... | Data: ...'.
- Dota 2 and League of Legends are independent games with no shared mechanics, items, abilities, or interactions. Entities from the two games CANNOT interact with each other under any circumstance.
- NEVER describe an interaction, comparison, synergy, counter, stacking behavior, or mechanical relationship between an entity from Dota 2 and an entity from League of Legends. Such interactions do not exist regardless of how the question is phrased.
- If the question asks about a relationship between entities from different games, do not construct one. Instead, describe each entity separately within its own game with explicit game attribution, or state that the entities belong to separate games and cannot interact.
- NEVER construct a claim about one game by inverting or contrasting a claim about the other game.
- This rule applies even when context chunks for both entities appear well-grounded and topically relevant. Per-entity grounding does not imply cross-entity interaction.
- Hedged or conditional framing is NOT permitted for cross-game relationships. Do not write phrases like 'if they interacted...', 'we can infer that...', or 'this implies that...' when the entities involved belong to different games.

REASONING:

Before giving your final answer, reason step by step through the context to extract the correct information. If the question asks about a specific level, rank, or tier, identify which positional value corresponds to the requested level. Do NOT show your reasoning process in the response. Only provide the final answer with the specific values and facts.

CONTEXT FORMAT GUIDE:

- The context comes from a structured JSON knowledge base. Each chunk begins with a header of the form 'Game: <game> | <entity_type>: <entity_name> | Section: <section> | Data: <content>'. The 'Game:' field identifies which game the chunk describes; the content after 'Data:' is the substantive information.

- Inside the Data content, you may encounter JSON syntax such as curly braces {}, square brackets [], and key-value pairs.
 - Square brackets [] denote lists. For example, a 'used_in' key with value ['Blade of the Ruined King', 'Wit's End'] means the item is used in building those two items.
 - Slash-separated values like '15 / 40 / 80 / 150' represent scaling values that increase or decrease with level. The leftmost value is the lowest level, and the rightmost value is the highest level (maximum rank of the ability).
 - A slash-separated sequence may end with a trailing unit word that applies to every value in the sequence, not just the last one. For example, '80 / 90 / 100 / 110 / 120 Mana' means each of the five values is a mana cost (80 mana at rank 1, 120 mana at rank 5, NOT '110' and a separate entity '120 Mana'). Similarly, '9 / 8 / 7 / 6 / 5 seconds' means each value is a duration in seconds. When extracting a specific value, strip the trailing unit and attach it to your chosen value (e.g., rank 5 of '9 / 8 / 7 / 6 / 5 seconds' is '5 seconds').
 - When asked about maximum level, or max rank, always use the RIGHTMOST value in the slash-separated sequence.
 - In League of Legends, champions have a maximum level of 18. If a question asks about level 18, treat it as maximum level and use the RIGHTMOST value.
 - In Dota 2, heroes have a maximum level of 30. If a question asks about level 30, treat it as maximum level and use the RIGHTMOST value.
 - When asked about level 1 or the base value, use the LEFTMOST value in the sequence.
- RESPONSE GUIDELINES:
- For each game you have chunks for, prefix your statements with the game name: 'In Dota 2, ...' or 'In League of Legends, ...'.
 - Be precise and cite specific values, names, and mechanics from the context.
 - When answering about a specific level, extract the correct positional value from slash-separated sequences.
 - Keep answers concise and factual, avoid speculation.
 - When referencing specific heroes, champions, items, or abilities that are NOT explicitly mentioned in the user's question, always frame them as examples using phrases like 'for example', 'such as', or 'e.g.'. Never present unasked-for specific entities as if they were the focus of the question.
 - Never use ellipsis (...) in your response. Always write complete sentences. Answer strictly from the provided context.

APPENDIX 4: EVALUATION RUBRIC AND EXAMPLES

Tier-2 Rubric

Table A4.1: Full Tier-2 Rubric

Dimension	Question	Options
Q1	Does the answer address the query?	○ Yes, fully ○ Partially, answers a related, but narrower question ○ No
Q2	Is the answer scoped to a specific game or games?	○ Covers both games with per-game attribution ○ Scoped to one game with explicit attribution ○ Scoped without any attribution ○ Mixes mechanics/terms from both games as if one game
Q3	Does any claim contradict the retrieved context?	○ No ○ Yes

Table A4.2: Tier-2 Scoring

Score	Condition
2	○ Q1: Yes AND ○ Q2: in {both-attributed, one-attributed} AND ○ Q3: No
1	○ Q1: Partial AND ○ Q2: in {both-attributed, one-attributed} AND ○ Q3: No
0	○ Q1: No OR ○ Q2: in {one-unattributed, mixes-mechanics} OR ○ Q3: Yes

Tier-2 Examples

Table A4.3: Fully Correct Example

Query	How many towers does each team have on the map in total, and how are they distributed across lanes?
Answer	In Dota 2, each team has a total of 11 towers, distributed across three lanes with three towers in each lane and two towers defending the Ancient. In League of Legends, each team has a total of 11 towers, with three towers in each of three lanes and two more towers defending the nexus.
Ground Truth	In both games, there are 11 towers distributed over the 3 lanes, and there are additionally 2 towers defending the main faction building.
Evaluation	Q1 – Yes, fully Q2 – Covers both games with per-game attribution Q3 - No Hence, the score = 2.

Table A4.4: Partially Correct Example

Query	What is the maximum number of wards a single player can place on the map at one time?
Answer	In League of Legends, a single player can have a maximum of three green wards on the map at one time.
Ground Truth (LoL Variant)	A player can place up to 3 Stealth Wards and 1 Control Ward at the same time.
Evaluation	Q1 - Partial (answers a narrower question, covers only one ward type);

	Q2 - one-game with explicit attribution; Q3 - not in contradiction with the context retrieved but the Control Ward information is missing. Hence, the score = 1.
--	---

Table A4.5: Failed Example

Query	How does the fountain at each team's spawn point damages who enters its area?
Answer	In Dota 2, the fountains at each team's spawn point do not damage enemies who enter their area. The context only describes the vision mechanics and blockers around the fountains but does not mention any damage effects.
Ground Truth	In both games, fountain damages enemy units who enters the area, the exact amount of damage dealt can be found via chunks.
Evaluation	Q1 - Yes Q2 - one-game with explicit attribution; Q3 – Contradicts with the chunk. Hence, the score = 0.

Tier-3 Rubric

Table A4.6: Full Tier-3 Rubric

Condition	Options
Score is 1 if the answer DOES at least one of the following:	○ Explicitly states the entities belong to different games or that the premise is invalid ○ Refuses to answer on the grounds that the context does not support it

	○ Describes each entity separately with per-game attribution, making the cross-game mismatch evident ○ Answer correctly for one entity with clear attribution and does not fabricate or speculate about the other
Score is 0 if the answer does at least one of the following:	○ Unhedged fabrication (“X counters Y”) ○ Hedged Speculation (“we can infer that X would affect Y”) ○ Conditional / hypothetical framing (“if they interacted, X would ...”) ○ Any construction of cross-domain reasoning regardless of the epistemic markers used to frame it

Tier-3 Examples

Table A4.7: Fully Correct Example

Query	How much bonus armor does the Vanguard shield provide to Baron Nashor when purchased from the side shop?
Answer	I cannot provide information about interactions between entities from different games. The Vanguard shield is from the game Dota 2, while Baron Nashor is from the game League of Legends.
Evaluation	The answer recognizes the invalid premise, names both entities with explicit per-game attribution, and constructs no interaction. Score = 1

Table A4.8: Failed Example

Query	How does Rubick’s Spell Steal interact with Syndra’s Unleashed Power when both ultimates are cast simultaneously?
Answer	In Dota 2, when both Rubick’s Spell Steal and Syndra’s Unleashed Power are cast simultaneously, the interaction is not explicitly mentioned in the provided context. However, the

	stolen ability travels as a projectile toward Rubick at a speed of 1500, and Rubick gains the new ability and loses the previously stolen ability as the projectile reaches him. This implies that the Spell Steal can interrupt and replace Syndra’s Unleashed Power if Rubick casts Spell Steal first.
Evaluation	Despite the hedging (“the interaction is not explicitly mentioned ...”), the answer constructs an interaction between a Dota 2 ability and a League of Legends ability: specifically that Spell Steal “can interrupt and replace” Unleashed Power. Score = 0.

Table A4.9: Failed Example

Query	What is the Netherveil Cloak’s magic resistance value and does it stack with Hood of Defiance?
Answer	In Dota 2, the Netherveil Cloak increases most heroes’ magic resistance to 40%. It stacks multiplicatively with other magic resistance sources, including the Hood of Defiance.
Evaluation	“Netherveil Cloak” is a made-up item that exists in neither corpus. The answer fabricates an entity, a numeric value, and a stacking interaction. Score = 0.

APPENDIX 5: FULL RESULT TABLES

Table A5.1: Success Rates

Variant	T1 (N=194)	T2 (N=72)	T2 if-refs	T3 (N=57)
Vanilla Top-5	64.9%	29.2%	60.4%	-
Vanilla Top-10	68.0%	44.4%	69.4%	-
Recency-NoReranker-Top-5	87.6%	63.2%	-	82.5%
Recency-NoReranker-Top-10	90.7%	69.4%	-	87.7%
Recency-Reranker-Top-5	93.3%	68.1%	-	80.7%
Recency-Reranker-Top-10	92.3%	69.4%	-	87.7%
Recency-Dynamic	94.8%	72.2%	-	93.0%
PR-NoReranker-Top-5	87.1%	65.3%	-	86.0%
PR-NoReranker-Top-10	91.8%	70.8%	-	86.0%
PR-Reranker-Top-5	93.8%	75.0%	-	91.2%
PR-Reranker-Top-10	93.3%	68.8%	-	86.0%
PR-Dynamic	95.4%	72.9%	-	94.7%

Table A5.2: RAGAS Scores for Tier-1. Median (Mean) per-Variant

Variant	Faithfulness	Recall	Precision	Correctness	Relevancy
Vanilla Top-5	1.00 (0.74)	1.00 (0.70)	0.78 (0.60)	0.64 (0.60)	0.89 (0.64)
Vanilla Top-10	1.00 (0.78)	1.00 (0.76)	0.61 (0.58)	0.73 (0.65)	0.89 (0.67)

Table A5.2 (continued): RAGAS Scores for Tier-1. Median (Mean) per-Variant

Recency-NoReranker-Top-5	1.00 (0.91)	1.00 (0.88)	0.83 (0.68)	0.82 (0.75)	0.93 (0.88)
Recency-NoReranker-Top-10	1.00 (0.92)	1.00 (0.96)	0.70 (0.66)	0.82 (0.77)	0.93 (0.90)
Recency-Reranker-Top-5	1.00 (0.93)	1.00 (0.96)	1.00 (0.94)	0.83 (0.79)	0.93 (0.88)
Recency-Reranker-Top-10	1.00 (0.92)	1.00 (0.96)	1.00 (0.94)	0.83 (0.77)	0.93 (0.89)
Recency-Dynamic	1.00 (0.92)	1.00 (0.96)	1.00 (0.95)	0.83 (0.78)	0.94 (0.90)
PR-NoReranker-Top-5	1.00 (0.89)	1.00 (0.88)	0.95 (0.69)	0.77 (0.74)	0.93 (0.88)
PR-NoReranker-Top-10	1.00 (0.94)	1.00 (0.96)	0.71 (0.67)	0.83 (0.79)	0.93 (0.89)
PR-Reranker-Top-5	1.00 (0.91)	1.00 (0.96)	1.00 (0.94)	0.83 (0.78)	0.93 (0.90)
PR-Reranker-Top-10	1.00 (0.92)	1.00 (0.96)	1.00 (0.94)	0.82 (0.78)	0.93 (0.89)
PR-Dynamic	1.00 (0.92)	1.00 (0.96)	1.00 (0.95)	0.83 (0.78)	0.94 (0.90)

Table A5.3: Retrieval Metrics for Tier-1 (means)

Variant	Hit@1	Hit@3	Hit@5	Hit@10	MRR
Vanilla Top-5	0.47	0.62	0.68	0.68	0.55
Vanilla Top-10	0.47	0.61	0.68	0.75	0.56
Recency-NoReranker-Top-5	0.49	0.69	0.85	0.94	0.63

Table A5.3 (continued): Retrieval Metrics for Tier-1 (means)

Recency-NoReranker-Top-10	0.50	0.70	0.85	0.94	0.63
Recency-Reranker-Top-5	0.81	0.92	0.93	0.94	0.87
Recency-Reranker-Top-10	0.81	0.92	0.93	0.94	0.87
Recency-Dynamic	0.81	0.92	0.93	0.94	0.87
PR-NoReranker-Top-5	0.50	0.70	0.85	0.94	0.63
PR-NoReranker-Top-10	0.50	0.70	0.85	0.94	0.63
PR-Reranker-Top-5	0.81	0.92	0.93	0.94	0.87
PR-Reranker-Top-10	0.81	0.92	0.93	0.94	0.87
PR-Dynamic	0.81	0.92	0.93	0.94	0.87

Table A5.4: Latency Distribution (in ms)

Variant	Layer	Mean	Median	P5	P95	Min	Max
Vanilla Top-5	Pipeline	18081	17390	16366	21759	15455	46151
Vanilla Top-10	Pipeline	17915	17675	16344	20581	15705	26966
Recency-NoReranker-Top-5	GPU Shuttle	14463	12505	12228	29835	12060	61102
Recency-NoReranker-Top-5	Pipeline	36017	30792	26333	63280	24604	99214

Table A5.4 (continued): Latency Distribution (in ms)

Recency- NoReranker- Top-10	GPU Shuttle	13622	13243	12294	16068	11948	25579
Recency- NoReranker- Top-10	Pipeline	43906	40028	30875	68943	25856	109104
Recency- Reranker-Top-5	GPU Shuttle	14854	13630	12708	17482	12196	95446
Recency- Reranker-Top-5	Pipeline	45727	41635	33707	70225	29215	134782
Recency- Reranker-Top- 10	GPU Shuttle	15698	14917	13235	22887	12676	29144
Recency- Reranker-Top- 10	Pipeline	54832	52485	39930	81339	32031	117117
Recency- Dynamic	GPU Shuttle	14736	13369	12357	23906	12117	50636
Recency- Dynamic	Pipeline	40258	37074	30567	60984	27453	91656
PR- NoReranker- Top-5	GPU Shuttle	13904	12461	12173	20885	11900	52961
PR- NoReranker- Top-5	Pipeline	37441	32764	27149	61551	23837	99694
PR- NoReranker- Top-10	GPU Shuttle	13024	12866	12121	14188	11836	24397

Table A5.4 (continued): Latency Distribution (in ms)

PR- NoReranker- Top-10	Pipeline	40473	36756	28647	62780	26236	120656
PR-Reranker- Top-5	GPU Shuttle	16160	14646	12544	22353	12094	79609
PR-Reranker- Top-5	Pipeline	47889	43324	33178	75490	29667	123844
PR-Reranker- Top-10	GPU Shuttle	15821	15152	13219	23204	12218	38335
PR-Reranker- Top-10	Pipeline	58263	55749	38708	87673	34222	136286
PR-Dynamic	GPU Shuttle	17118	15358	12438	29575	12154	80090
PR-Dynamic	Pipeline	43706	40418	30666	72028	28195	136932